

SentraUjian — PRD v2

Daftar Isi

SentraUjian

1. Ringkasan & Tujuan Aplikasi
2. Batasan Pembuatan Sistem (Versi Awal MVP)
 - Yang Dikerjakan:
 - Yang Tidak Dikerjakan di Versi Awal:
3. Daftar Halaman & Struktur Menu (Pages & Routing)
 - A. Public & Autentikasi
 - B. Central Web — Super Admin & Content Admin (Dashboard Pusat)
 - C. Edge Admin Web — School Admin / School Operator / Proctor (Dashboard Sekolah)
 - D. Mobile React Native — Operator Sekolah
 - E. SentraExam Client — Aplikasi Desktop Peserta Milik Sendiri
4. Pedoman UI/UX & Design System
5. Pembagian Hak Akses Pengguna
6. Alur Kerja dan Fitur Utama
 - A. Pengelolaan & Review Bank Soal
 - B. Pembuatan & Versioning Paket Ujian
 - C. Distribusi Paket ke Edge Sekolah (1.000 sekolah)
 - D. Hari Ujian (Koreksi Otomatis di Edge & Skor Instan)
 - E. Sinkronisasi Hasil Edge → Pusat (Idempotent & Tahan Putus)
 - F. Monitoring Edge Realtime (Heartbeat & Distribusi)
 - G. Autentikasi & Keamanan
 - H. SentraExam Client — Aplikasi Ujian Milik Sendiri
7. Alur Navigasi & Arsitektur Layout
 - Arsitektur Layout (Persisten)
 - Bagan Alur (Flowchart)
8. Kebutuhan Non-Fungsional (SEO, Keamanan, & Performa)
9. Panduan Bahasa, Copywriting, & Data Dummy
10. Fondasi Teknis (Untuk Tim Pengembang / Programmer & AI)
 - Struktur Skema Database Nyata (Central PostgreSQL — Drizzle-style TypeScript)
 - Struktur Skema Database Nyata (Edge PostgreSQL — Drizzle-style TypeScript)
 - Struktur Tabel Tambahan (Peserta & Token Sekali Pakai)
 - Variabel Lingkungan (`.env.example`)
11. Struktur Tim, Ownership, dan Pola Kolaborasi
 - A. Susunan Tim
 - B. Batas Ownership Teknis
 - C. Matriks RACI Ringkas
 - D. Kontrak Antar-Tim
 - E. Jalur Dependensi Pekerjaan
 - F. Ritme Eksekusi dan Serah Terima

12. Tahapan Pengerjaan & Task Breakdown (Actionable Work Breakdown Structure)

Tahap 1: Fondasi Monorepo, Kontrak, UI/UX Dummy (Central Web, Edge Admin Web, Mobile, SentraExam Client)

Tahap 2: Backend Go, Database, Auth, Distribusi, Edge MVP, Sync Idempotent

Tahap 3: Hardening Keamanan, Automated Tests, Docker Compose, Load Test, Deployment, Dokumentasi

13. Master Starter Prompt (Siap Coding untuk AI Agent)

1. Ringkasan & Tujuan Aplikasi

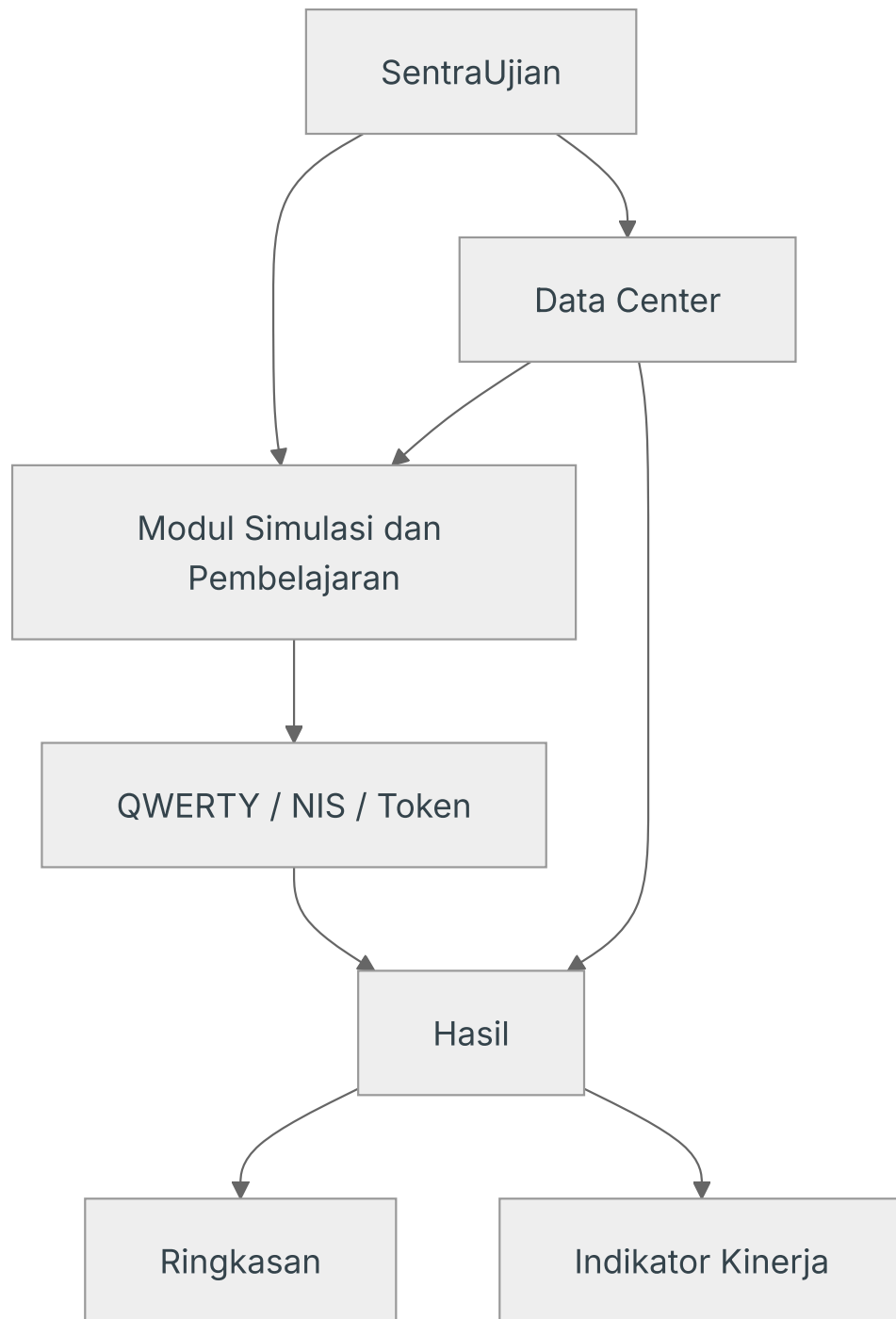
Bagian ini menjelaskan gambaran umum proyek agar dipahami bersama oleh pemilik ide/klien dan tim pengembang.

- **Nama Aplikasi:** SentraUjian
- **Penjelasan Singkat:** Platform simulasi ujian sekolah nasional berbasis arsitektur dua lapis (central-server & edge-server) yang dirancang untuk melayani hingga 1.000 sekolah secara mandiri — ujian tetap berjalan 100% di jaringan lokal (LAN) sekolah meski internet terputus, lalu hasilnya disinkronkan ke server pusat secara aman dan idempoten.
- **Masalah yang Diselesaikan:**
 - Ujian sekolah sering gagal total ketika koneksi internet terputus di tengah ujian.
 - Server pusat tidak mampu menampung ratusan ribu peserta jika mereka semua terhubung langsung secara bersamaan.
 - Kunci jawaban sering bocor karena dikirim ke komputer peserta.
 - Distribusi paket soal ke banyak sekolah tidak terkontrol, tidak dapat diaudit, dan rawan duplikasi data.
 - Rekap hasil nasional manual memakan waktu lama dan rawan salah hitung.
- **Pengguna Aplikasi:**
 - **Super Admin Pusat:** mengelola sekolah, paket soal, distribusi, perangkat edge, kebijakan sistem, dan rekap hasil nasional.
 - **Content Admin:** mengelola bank soal, mengunggah materi mentah (PDF/DOCX/JSON/XLSX/gambar), dan mengelola prediksi soal.
 - **Reviewer:** mereview soal **AI_DRAFT** sebelum dipromosikan menjadi **APPROVED**.
 - **School Admin:** mengelola akun operator sekolah, jadwal ujian, dan konfigurasi paket yang diterima.
 - **School Operator:** mengoperasikan ujian hari-H (aktivasi sesi, monitoring peserta, sinkronisasi hasil).
 - **Proctor (Pengawas):** memantau peserta via LAN secara realtime.
 - **Student (Peserta):** mengerjakan ujian memakai token sekali pakai melalui aplikasi SentraExam Client.
 - **Auditor:** membaca jejak audit dan mengevaluasi kepatuhan sistem.
- **Target Keberhasilan:**
 - Ujian dapat berlangsung di 1.000 sekolah tanpa ketergantungan koneksi internet.
 - Distribusi 1 paket soal hanya didownload sekali per edge sekolah (bukan per siswa).
 - Hasil ujian tersinkronisasi otomatis ke pusat dengan tingkat duplikasi 0% dan tingkat kehilangan data 0%.
 - Waktu koreksi otomatis di edge < 3 detik setelah peserta menekan submit.
 - Jawaban benar (answer key) tidak pernah muncul di payload yang diterima komputer peserta.
 - Uptime central server ≥ 99,5% dengan readiness check & graceful degradation.

PRD v2.1.0 Penyempurnaan berdasarkan hasil perumusan kebutuhan dan arsitektur yang telah disepakati.

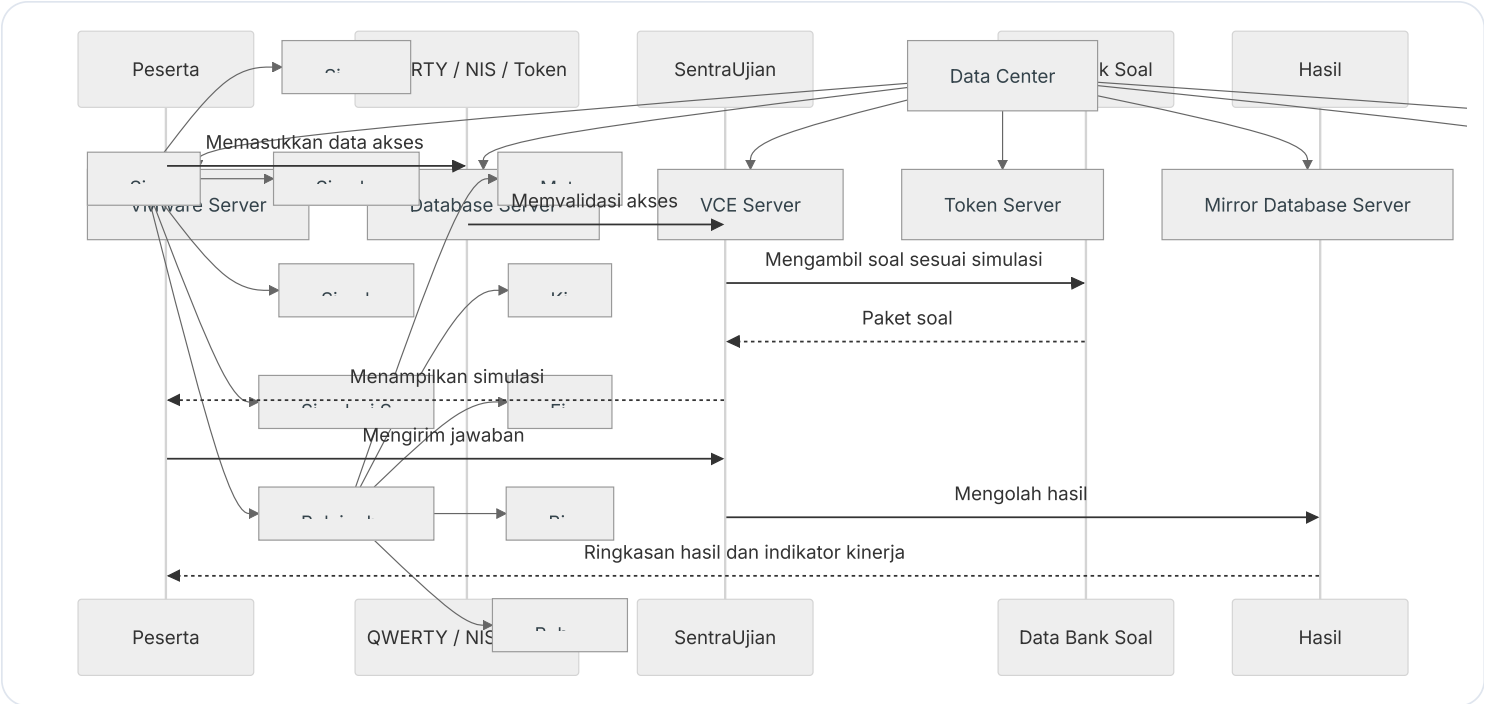
2. Baseline Produk dan Arsitektur

PRD v2.1 mempertahankan struktur dan label dari hasil pembahasan sebagai baseline produk. Diagram berikut menjadi peta utama hubungan antara Data Center, modul simulasi, akses peserta, dan hasil.



2.1 Komponen Data Center

Komponen berikut dicatat dengan nama yang sama seperti baseline pembahasan. PRD tidak memperluas atau mengganti istilah **VCE Server** maupun **QWERTY** di luar label yang telah disepakati.

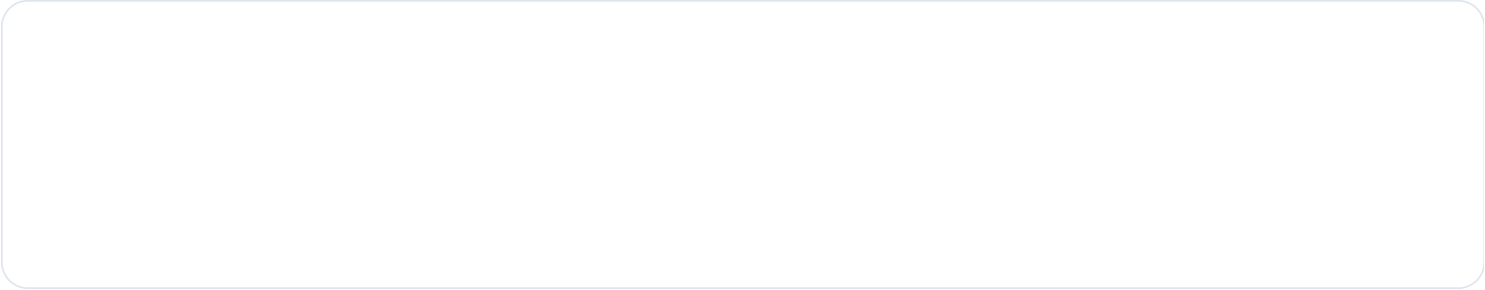


Label baseline	Posisi dalam model produk
VMware Server	Komponen infrastruktur Data Center.
Database Server	Penyimpanan data platform dan transaksi.
VCE Server	Komponen Data Center dengan nama baseline tetap.
Token Server	Komponen pengelolaan token pada Data Center.
Mirror Database Server	Komponen mirror database pada Data Center.
Data Bank Soal	Sumber bank soal utama.
Data Bank Soal Prediksi	Sumber bank soal prediksi.

3. Modul Simulasi dan Pembelajaran

- Jenis simulasi menjadi kategori layanan yang dapat dipilih sesuai kebutuhan assessment.
- Belajar berdasarkan modul menjadi jalur pembelajaran berbasis materi.
- Bank soal dan bank soal prediksi menjadi sumber konten yang dikelola melalui Data Center.
- Setiap aktivitas menghasilkan data yang dapat dirangkum dalam Ringkasan dan Indikator Kinerja.

4. Alur Akses dan Hasil



PRD v2.1 mempertahankan label **QWERTY / NIS / Token** sebagai bagian dari baseline akses. Rincian field, validasi, hak akses, dan kebijakan keamanan mengikuti konfigurasi implementasi yang disetujui untuk setiap jenis simulasi.

5. Keluaran Produk

Keluaran	Isi
Ringkasan	Ikhtisar aktivitas dan hasil peserta atau kelompok sesuai konteks layanan.
Tujuan	Arah penggunaan simulasi dan pembelajaran yang ditetapkan dalam layanan.
Indikator Kinerja	Ukuran performa yang dihitung dari aktivitas dan hasil.
Hasil	Output pengerjaan simulasi yang tersimpan dan dapat diproses sesuai aturan.

6. Ketentuan Perubahan v2.1

- v2.1 menambahkan baseline produk dan arsitektur dari hasil pembahasan yang telah disepakati.
- Label **VMware Server**, **Database Server**, **VCE Server**, **Token Server**, **Mirror Database Server**, **Data Bank Soal**, **Data Bank Soal Prediksi**, **QWERTY / NIS / Token**, dan jenis simulasi dipertahankan.
- Perubahan ini tidak menghapus, mengganti, atau menimpa PRD v2.0.0 maupun PRD v1.0.0.

7. Batasan Pembuatan Sistem (Versi Awal MVP)

Menegaskan fitur apa yang dikerjakan di versi awal dan apa yang sengaja ditunda agar aplikasi cepat selesai dan tidak membengkak (mencegah scope creep).

Yang Dikerjakan:

- Monorepo Turborepo dengan `apps/` dan `packages/` sesuai blueprint: `central-web`, `central-api`, `central-worker`, `edge-server`, `edge-admin-web`, `exam-client`, dan aplikasi mobile React Native pendamping operator (`edge-operator-mobile`).
- Central API (Go + Gin/Fiber) lengkap dengan OpenAPI 3.1 dan endpoint minimal sesuai Bab 6 PRD.
- Edge Server (Go single binary) berjalan di Windows & Ubuntu, mendukung PostgreSQL lokal dan SQLite WAL (prototipe).
- Autentikasi peserta & operator via **Token Ujian Sekali Pakai** (single-use exam token) yang diterbitkan edge server.
- Distribusi paket soal dengan resumable download, envelope encryption (RSA-OAEP wrap + AES-GCM content), checksum, dan signature verification.

- **SentraExam Client buatan sendiri** untuk komputer peserta, dibangun sebagai aplikasi desktop lintas platform memakai Tauri 2 + React. Client mencakup launcher, login token, pemeriksaan koneksi LAN, halaman ujian, navigasi soal, timer resmi edge, autosave, pemulihan sesi, submit, dan hasil.
- Protokol komunikasi milik SentraUjian antara SentraExam Client dan Edge Server melalui REST + WebSocket di LAN, dengan kontrak terversi, device/session binding, dan tanpa ketergantungan pada aplikasi ujian pihak ketiga.
- Result Outbox di edge dengan idempotency key, exponential backoff, dan checkpoint batch.
- Dashboard Super Admin: Manajemen Sekolah & Edge, Bank Soal & Review **AI_DRAFT**, Package Builder & Distribusi, Rekap Hasil Nasional, Audit Log, Monitoring Heartbeat.
- Dashboard Operator Sekolah: Aktivasi Ujian & Jadwal, Monitor Peserta Realtime via LAN (WebSocket), Unduh & Verifikasi Paket, Sinkronisasi Hasil ke Pusat, Status Storage & Log Operasional.
- Koreksi otomatis di edge + tampilan skor instan.
- **In-App Notification Edge** (toast/panel notifikasi di UI, dipublish via WebSocket dari edge, disimpan di tabel **edge_notifications**).
- Sinkronisasi hasil edge→pusat secara idempotent dan tahan putus koneksi.
- Docker Compose untuk development dan deployment awal (central & edge).
- Automated tests (Go: unit + integration; React: Vitest + Testing Library; E2E: Playwright).
- Seed data 3 sekolah simulasi (**SMAN 1 Nusantara**, **SMKN 2 Bahari**, **SMA Tunas Bangsa**) beserta soal & paket contoh.

❑ **Yang Tidak Dikerjakan di Versi Awal:**

- Integrasi ke aplikasi ujian milik vendor/pihak ketiga. Seluruh pengalaman ujian peserta dibangun dan dimiliki sendiri sebagai **SentraExam Client**.
- Model ML/DL pipelines (hanya menyediakan kontrak API, worker skeleton, dan tabel **prediction_runs** siap pakai).
- Data warehouse terpisah (BigQuery/ClickHouse); rekap nasional awal memakai materialized view di PostgreSQL pusat.
- Anti-cheating tingkat lanjut seperti proctoring kamera AI, deteksi wajah, dan screen-lock OS.
- Multi-tenant billing, langganan sekolah, dan invoice otomatis.
- SSO Google/Microsoft untuk sekolah; hanya kredensial lokal + token sekali pakai.
- Mobile app peserta penuh (fokus React Native hanya untuk aplikasi operator sekolah mobile).

8. Daftar Halaman & Struktur Menu (Pages & Routing)

Daftar lengkap halaman yang harus dibuat, dikelompokkan berdasarkan area atau peran pengguna (Role).

A. Public & Autentikasi

- **/login** (Login Pusat): Form login untuk seluruh staf pusat (Super Admin, Content Admin, Reviewer).
- **/login/sekolah** (Login Sekolah): Form login School Admin & School Operator ke edge lokal.
- **/forgot-password** (Lupa Password): Pemulihan akun staf pusat.
- **/health** (Health Check Publik): Endpoint statis untuk uptime monitoring.

B. Central Web — Super Admin & Content Admin (Dashboard Pusat)

- **/dashboard** (Overview Nasional): Kartu ringkasan jumlah sekolah, edge online/offline, jadwal ujian aktif, distribusi sedang berjalan, dan heatmap nasional.
- **/schools** (Daftar Sekolah): Tabel sekolah dengan filter provinsi, kota, jumlah peserta, dan status.
- **/schools/new** (Tambah Sekolah): Form registrasi sekolah baru + generate kunci edge device.
- **/schools/:id** (Detail Sekolah): Profil sekolah, daftar edge device, heartbeat terakhir, riwayat distribusi.
- **/edges** (Daftar Edge Device): Tabel edge, versi aplikasi, status koneksi Tailscale, last heartbeat.

- `/edges/:id` (Detail Edge): Log heartbeat, paket yang terpasang, kunci publik, riwayat sinkronisasi.
- `/question-bank` (Bank Soal Terverifikasi): Tabel soal `APPROVED`, `DRAFT`, `IN_REVIEW`.
- `/question-bank/prediction` (Bank Soal Berbasis Prediksi): Soal `AI_DRAFT` dengan confidence score, model version, dan tombol ke review.
- `/question-bank/import` (Import Soal): Upload JSON/CSV/XLSX/PDF/DOCX + gambar massal, dengan validator hasil validasi per baris.
- `/question-bank/:id` (Detail Soal): Preview soal, kunci jawaban terenkripsi, pembahasan, kisi-kisi, riwayat review.
- `/reviews` (Review Soal): Antrean soal `AI_DRAFT` / `IN_REVIEW` untuk Reviewer.
- `/package-builder` (Daftar Paket): Daftar paket + versi.
- `/package-builder/new` (Buat Paket): Wizard 4 langkah (Pilih Soal → Konfigurasi → Preview → Publish).
- `/package-builder/:id` (Detail Paket): Manifest, checksum, signature, versi, form distribusi.
- `/distributions` (Monitoring Distribusi): Progress bar per sekolah, status (Menunggu/Diunduh/Berhasil/Gagal/Offline).
- `/schedules` (Jadwal Ujian): Kalender & tabel jadwal lintas sekolah.
- `/results` (Rekap Hasil Nasional): Filter sekolah, subtes, tanggal; ekspor CSV/XLSX.
- `/sync-failures` (Sinkronisasi Gagal): Batch yang gagal beserta aksi retry manual.
- `/audit-log` (Audit Log): Filter action, actor, entity, timestamp.
- `/settings/ml` (Pengaturan Model ML/DL): Daftar model, versi, dataset version, status aktif/nonaktif.
- `/profile` (Profil Admin): Ganti kata sandi, konfigurasi 2FA (opsional).

C. Edge Admin Web — School Admin / School Operator / Proctor (Dashboard Sekolah)

- `/local/dashboard` (Status Server Lokal): Info versi edge, IP LAN, uptime, CPU/RAM/Disk.
- `/local/network` (Status LAN): Daftar klien di LAN, bandwidth, status konektivitas LAN.
- `/local/packages` (Paket Tersedia): Tabel paket terverifikasi, checksum, dan status aktivasi.
- `/local/schedules` (Aktivasi Ujian & Jadwal): Panel aktivasi sesi ujian, tombol start/stop, aturan waktu.
- `/local/participants` (Daftar Peserta): Import peserta lokal, generate token sekali pakai, cetak kartu token.
- `/local/monitor` (Monitor Peserta Realtime): Grid live (WebSocket) menampilkan status peserta: aktif/selesai/terputus.
- `/local/sync` (Sinkronisasi Hasil): Ringkasan antrean outbox, tombol sinkron manual, riwayat batch.
- `/local/storage` (Kapasitas Storage): Grafik disk, peringatan jika hampir penuh.
- `/local/logs` (Log Operasional): Log lokal terstruktur + filter level.
- `/local/notifications` (In-App Notification Edge): Panel notifikasi distribusi paket, status ujian, sinkronisasi.
- `/local/settings` (Pengaturan Edge): Konfigurasi waktu, token expiry, rate limit.

D. Mobile React Native — Operator Sekolah

- `/mobile/login` (Login Operator): Login token sekali pakai via aplikasi mobile.
- `/mobile/dashboard` (Ringkasan Mobile): Kartu status edge, sesi aktif, jumlah peserta.
- `/mobile/monitor` (Monitor Mobile): Daftar peserta realtime via WebSocket.
- `/mobile/notifications` (Notifikasi Mobile): Push-notifikasi untuk event penting edge.

E. SentraExam Client — Aplikasi Desktop Peserta Milik Sendiri

- `/client/` (Launcher SentraExam Client): Halaman awal aplikasi desktop peserta, menampilkan status koneksi edge, versi client, identitas perangkat, dan tombol mulai.
- `/client/login` (Login Peserta): Input token sekali pakai.
- `/client/exam` (Layar Ujian): Soal + navigasi nomor + timer resmi edge.
- `/client/result` (Skor Instan): Menampilkan skor setelah submit (koreksi edge).
- `/client/system-check` (Pemeriksaan Perangkat): Memeriksa versi aplikasi, koneksi LAN, waktu edge, resolusi layar, penyimpanan lokal, dan status sesi sebelum ujian.
- `/client/reconnect` (Pemulihan Sesi): Memulihkan attempt yang sah setelah aplikasi tertutup, komputer restart, atau koneksi LAN terputus sementara.

- `/client/locked` (Mode Ujian): Tampilan fullscreen terkontrol untuk mengurangi perpindahan aplikasi; bukan pengganti proctoring OS tingkat lanjut.

9. Pedoman UI/UX & Design System

Panduan visual konkret agar AI coding assistant tidak membuat UI yang kaku atau default.

- **Skema Warna:** Tema profesional "Institutional Blue" yang tenang, cocok untuk pengawasan lama tanpa membuat mata lelah.
 - Primary: `HSL(215, 82%, 42%)` — biru institusional utama (tombol aksi, navbar).
 - Primary Foreground: `HSL(0, 0%, 100%)`.
 - Secondary: `HSL(210, 30%, 94%)` — abu muda untuk panel sekunder.
 - Accent: `HSL(160, 68%, 40%)` — hijau untuk status "Berhasil/Tersinkron".
 - Warning: `HSL(38, 92%, 50%)` — kuning untuk "Menunggu/Proses".
 - Danger: `HSL(0, 78%, 52%)` — merah untuk "Gagal/Offline".
 - Background: `HSL(210, 20%, 98%)`.
 - Foreground: `HSL(215, 30%, 14%)`.
 - Border: `HSL(214, 20%, 88%)`.
 - Dark Mode: Background `HSL(215, 28%, 10%)`, Surface `HSL(215, 24%, 14%)`, Primary dicerahkan menjadi `HSL(215, 82%, 60%)`.
 - **Tipografi:**
 - Heading: **Geist Sans** (fallback **Inter**) — bobot 600/700.
 - Body: **Inter** — ukuran 14–16px, line-height 1.55.
 - Monospace (untuk UUID, checksum, token): **JetBrains Mono**.
 - Skala: `text-xs` (12) → `text-sm` (14) → `text-base` (16) → `text-lg` (18) → `text-xl` (20) → `text-2xl` (24) → `text-3xl` (30).
 - **Aturan Komponen:**
 - Radius: `rounded-md` (6px) untuk input, `rounded-lg` (10px) untuk tombol, `rounded-xl` (14px) untuk kartu & modal.
 - Bayangan: kartu default `shadow-sm`, kartu interaktif `hover:shadow-md`, modal `shadow-2xl`.
 - Tombol: tinggi minimum 40px, padding horizontal `px-4`, ikon dari **Lucide**.
 - Tabel: zebra-stripping tipis (`even:bg-muted/40`), sticky header, kolom status memakai `<Badge>` berwarna konsisten.
 - Form: label di atas input, error text merah di bawah input, semua field divalidasi `react-hook-form` + `zod`.
 - Loading: skeleton blocks (bukan spinner full-page) agar UI tetap terasa responsif.
 - Notifikasi: sistem `sonner` toast di kotak kanan atas, khusus edge juga muncul di panel `/local/notifications` (In-App Notification Edge).
 - **Nuansa & Vibe:**
 - Bersih, formal, dense-data (cocok untuk dashboard monitoring ratusan sekolah).
 - Minim animasi — hanya mikro-transisi 150ms pada hover/focus, tanpa parallax atau transisi dekoratif.
 - Palet tenang (hindari warna neon), huruf jelas, kontras WCAG AA.
 - State penting (Online/Offline/Gagal) selalu ditandai ikon + warna + teks (bukan hanya warna).
 - Aksesibilitas: focus ring `ring-2 ring-primary/60`, semua kontrol dapat diakses keyboard.
-

10. Pembagian Hak Akses Pengguna

Tabel hak akses yang menentukan siapa saja yang boleh melihat, mengedit, atau mengelola data.

Menu / Halaman	Publik	SUPER_ADMIN	CONTENT_ADMIN	REVIEWER	SCHOOL_ADMIN	SCHOOL_OPERATOR	PROCTOR	STUDENT	AUDITOR
/login & /login/sekolah	☐	☐	☐	☐	☐	☐	☐	☐	☐
/dashboard (Overview Nasional)	☐	☐	☐	☐	☐	☐	☐	☐	☐
/schools & /edges	☐	☐	☐	☐	☐	☐	☐	☐	☐
/question-bank (CRUD)	☐	☐	☐	👁️ Lihat	☐	☐	☐	☐	👁️ Lihat
/reviews (Review AI_DRAFT)	☐	☐	☐	☐	☐	☐	☐	☐	👁️ Lihat
/package-builder	☐	☐	☐	👁️ Lihat	☐	☐	☐	☐	👁️ Lihat
/distributions	☐	☐	☐	☐	☐	☐	☐	☐	☐
/results (Rekap Nasional)	☐	☐	☐	☐	☐ (sekolah sendiri)	👁️ Lihat (sekolah sendiri)	☐	👁️ Skor sendiri	☐
/audit-log	☐	☐	☐	☐	☐	☐	☐	☐	☐
/settings/ml	☐	☐	☐	☐	☐	☐	☐	☐	👁️ Lihat
/local/dashboard & /local/network	☐	☐	☐	☐	☐	☐	☐	☐	👁️ Lihat
/local/packages	☐	☐	☐	☐	☐	☐	👁️ Lihat	☐	👁️ Lihat
/local/schedules (Aktivasi)	☐	☐	☐	☐	☐	☐	☐	☐	☐
/local/participants (Token)	☐	☐	☐	☐	☐	☐	👁️ Lihat	☐	☐
/local/monitor	☐	☐	☐	☐	☐	☐	☐	☐	👁️ Lihat
/local/sync & /local/storage	☐	☐	☐	☐	☐	☐	☐	☐	👁️ Lihat
/local/logs	☐	☐	☐	☐	☐	☐	☐	☐	👁️ Lihat
/local/notifications	☐	☐	☐	☐	☐	☐	☐	☐	☐
/client/login & /client/exam (Token Sekali Pakai)	☐	☐	☐	☐	☐	☐	☐	☐ (token sendiri)	☐
/client/result (Skor Instan)	☐	☐	☐	☐	☐	☐	☐	☐ (milik sendiri)	👁️ Lihat

Catatan: 👁️ Lihat berarti hanya dapat membaca tanpa hak ubah/hapus. Semua akses di atas ditegakkan di dua lapis: middleware Go (RBAC) di API dan route guard di React TanStack Router.

11. Alur Kerja dan Fitur Utama

Menjelaskan cara kerja setiap fitur utama dalam bahasa yang mudah dipahami serta aturan logikanya.

A. Pengelolaan & Review Bank Soal

1. Cara Kerja:

- Content Admin membuka `/question-bank/import` dan mengunggah berkas mentah (gambar, PDF/DOCX, JSON/CSV/XLSX, kunci jawaban, pembahasan, kisi-kisi).
- Sistem memvalidasi tiap entri: jumlah opsi, tipe jawaban, panjang teks, keberadaan gambar, dan format kunci.
- Soal tersimpan pada salah satu kategori: **Bank Soal Terverifikasi** (`DRAFT` / `IN_REVIEW` / `APPROVED` / `REJECTED` / `ARCHIVED`) atau **Bank Soal Berbasis Prediksi** (`AI_DRAFT`).
- Soal `AI_DRAFT` (dari ML/DL/LLM) wajib melewati halaman `/reviews` sebelum berubah status.
- Hanya soal berstatus `APPROVED` yang boleh dipakai di `/package-builder`.

2. Aturan Sistem:

- Batas ukuran file: gambar ≤ 10 MB, PDF/DOCX ≤ 50 MB, XLSX/CSV ≤ 20 MB.
- Setiap gambar disimpan di S3/MinIO dengan `content_hash` (SHA-256) untuk deduplikasi.
- Status tidak boleh melompat (contoh: dari `AI_DRAFT` tidak bisa langsung ke `APPROVED`).
- Reviewer wajib mengisi `decision_reason` saat menolak soal.
- Tidak menggunakan kosakata "bocoran"/"prediksi pasti"; selalu gunakan istilah **"soal latihan berbasis prediksi"**.
- Semua perubahan entri soal dicatat pada `audit_logs`.

B. Pembuatan & Versioning Paket Ujian

1. Cara Kerja:

- Admin membuka `/package-builder/new` memilih soal `APPROVED` dari subtes (`PU` , `PPU` , `PBM` , `PK` , `LBI` , `LBE` , `PM`).
- Sistem menyusun paket immutable dengan struktur `manifest.json` , `questions.json` , `assets/*` , `answer-key.enc` , `explanations.enc` , `scoring.json` , `checksums.json` , `signature.sig` .
- Admin menekan **Publish** → central-worker membangun arsip, menghitung signature (Ed25519), dan menyimpan di S3/MinIO.
- Setiap revisi wajib menghasilkan versi baru (`v1` , `v2` , ...). Paket lama hanya bisa diarsipkan (`ARCHIVED`).

2. Aturan Sistem:

- Envelope Encryption**: konten dienkripsi dengan AES-256-GCM memakai Data Encryption Key (DEK); DEK dibungkus RSA-OAEP memakai public key edge (`edge_device_keys.public_key`).
- Setiap paket menyertakan `min_edge_version` dan `immutable=true` .
- Sistem menolak paket jika ada subtes yang kurang soal atau kunci/pembahasan kosong.
- Paket yang sudah dipublish tidak dapat diubah field apapun; perubahan wajib membuat versi baru.

C. Distribusi Paket ke Edge Sekolah (1.000 sekolah)

1. Cara Kerja:

- Admin memilih target: sekolah A/B/C/D atau **semua sekolah** di `/package-builder/:id` .
- Central API membuat `package_distributions` + `distribution_events` .
- Edge agent melakukan **pull** via Tailscale dengan **resumable download** (HTTP Range).
- Edge memverifikasi `signature.sig` (Ed25519 dengan public key pusat), `checksums.json` , dan kompatibilitas versi.
- Jika lolos, paket disimpan di `package_cache` + `package_files` ; status → `BERHASIL` .
- Jika gagal → status `VERIFIKASI_GAGAL` , `VERSI_TIDAK_KOMPATIBEL` , atau `PERANGKAT_OFFLINE` .
- Edge mengirim `POST /api/v1/edges/{id}/packages/{packageId}/acknowledge` .

2. Aturan Sistem:

- Server pusat **hanya mengirim satu salinan** paket ke setiap edge sekolah — bukan per siswa.
- Distribusi paralel menggunakan worker pool + Redis distributed lock per sekolah.
- Retry exponential backoff: 30s → 2m → 10m → 1h, maksimum 12 kali, lalu masuk antrean `FAILED_PERMANENT` yang bisa diretry manual.
- Semua event dicatat di `distribution_events` (bisa diputar ulang untuk audit).

D. Hari Ujian (Koreksi Otomatis di Edge & Skor Instan)

1. Cara Kerja:

- Operator sekolah membuka `/local/schedules`, memilih paket `BERHASIL` dan menekan **Aktifkan Sesi**.
- Edge membuat `local_exam_sessions` dan generate **Token Ujian Sekali Pakai** untuk tiap peserta; token dicetak/diunduh via `/local/participants`.
- Peserta membuka aplikasi SentraExam Client (via LAN `http://ujian.local`) dan memasukkan token.
- Edge memvalidasi token (single-use, expiry, exam session aktif), klaim token, dan membuat `local_attempts` dengan UUID global (ULID).
- Edge mengirim soal **tanpa kunci jawaban** — hanya teks/gambar opsi.
- Autosave tiap jawaban ke `local_attempt_answers` (debounced 2 detik).
- Timer resmi dikendalikan edge server** (bukan jam komputer peserta); peserta tidak dapat mengubah waktu.
- Saat peserta klik submit, edge mendekripsi `answer-key.enc` **hanya di memori**, melakukan koreksi otomatis, dan menyimpan `local_results` dengan skor instan.
- Hasil masuk ke `result_outbox` untuk disinkronkan ke pusat.
- Panel `/local/monitor` (WebSocket) menampilkan realtime: peserta **aktif, selesai, terputus**.

2. Aturan Sistem:

- Answer key, pembahasan terkunci, scoring secret, dan token perangkat edge DILARANG dikirim ke komputer peserta.**
- Token kedaluwarsa mengikuti kebijakan operator (default 60 menit sebelum sesi dimulai sampai 5 menit setelah sesi berakhir).
- Attempt setelah final submit **immutable** (`locked_at`).
- Jika edge restart saat ujian: state pulih dari PostgreSQL WAL; timer melanjutkan dari `started_at` yang tersimpan.
- Jika peserta refresh: state attempt dipulihkan dari `local_attempt_answers`.
- In-App Notification Edge**: notifikasi toast muncul di dashboard edge saat paket selesai diunduh, sesi dimulai, peserta selesai, dan sinkronisasi sukses/gagal.

E. Sinkronisasi Hasil Edge → Pusat (Idempotent & Tahan Putus)

1. Cara Kerja:

- Setiap 60 detik (dan saat event selesai), edge agent membaca `result_outbox`.
- Edge mengirim batch (default maks. 200 attempt) via `POST /api/v1/sync/results` dengan `batch_id` (ULID) dan `idempotency_key` per attempt.
- Pusat menyimpan `sync_batches` dan `sync_events`; attempt duplikat diabaikan tanpa menghapus data yang sudah ada.
- Setelah ACK, edge menandai outbox sebagai `SYNCED`, dan meng-update `sync_checkpoints`.
- Gagal → retry exponential backoff 5s → 30s → 2m → 10m → 1h; data **tidak boleh hilang** (outbox persisten).

2. Aturan Sistem:

- Pemetaan konsep tegas: `answer_key` = kunci dari paket; `student_response` = jawaban peserta; `exam_result` = hasil penilaian.
- Pusat menolak duplikasi **tanpa** menghapus data (`ON CONFLICT DO NOTHING`).
- Batch yang gagal permanen tercatat di `/sync-failures` untuk retry manual.
- Hasil seluruh sekolah di-roll-up ke materialized view `central_results_mv` untuk rekap nasional.

F. Monitoring Edge Realtime (Heartbeat & Distribusi)

1. Cara Kerja:

- Setiap edge mengirim `POST /api/v1/edges/heartbeat` setiap 30 detik berisi versi aplikasi, uptime, storage, jumlah peserta aktif.
- Super Admin melihat status di `/edges` dan `/distributions` (Menunggu, Diunduh, Berhasil, Gagal, Versi tidak kompatibel, Offline).
- Jika tidak ada heartbeat > 3 menit → status `OFFLINE` + **In-App Notification Edge** ke pusat.

2. Aturan Sistem:

- Heartbeat ditandatangani HMAC dengan `edge_secret`.
- Replay protection via `nonce` + timestamp (±2 menit).
- Riwayat heartbeat disimpan 90 hari di `edge_heartbeats`.

G. Autentikasi & Keamanan

1. Cara Kerja:

- Login staf pusat via `POST /api/v1/auth/login` → access token JWT 15 menit + refresh token 7 hari.
- Login sekolah via `POST /local/v1/auth/login` → token lokal JWT 30 menit.
- Peserta **hanya** login via token sekali pakai.
- Mutual authentication edge↔pusat memakai mTLS (Tailscale certs) + signature HMAC.

2. Aturan Sistem:

- Password memiliki minimum 12 karakter, dicek terhadap list password umum.
- Setiap edge punya keypair unik (`edge_device_keys`); rotasi tiap 90 hari.
- Rate limit: 20 request/menit/IP untuk `/auth/login`, 600/menit untuk endpoint data.
- Server time = sumber waktu resmi; jam klien disinkron di setiap request attempt.
- Semua perubahan paket dan kunci dicatat di `audit_logs` (immutable append-only).

H. SentraExam Client — Aplikasi Ujian Milik Sendiri

1. Cara Kerja:

- Tim membangun `apps/exam-client` sebagai aplikasi desktop Tauri 2, bukan membungkus atau memanggil produk ujian pihak ketiga.
- Saat pertama dibuka, client mencari edge melalui alamat yang dikonfigurasi operator, DNS lokal `ujian.local`, atau discovery mDNS yang dapat dimatikan.
- Halaman `system-check` memverifikasi versi minimum, koneksi LAN, waktu resmi edge, ruang penyimpanan, resolusi, dan status sesi.
- Peserta memasukkan token sekali pakai. Edge mengikat token ke `client_instance_id`, peserta, sesi, dan attempt dalam satu transaksi.
- Client menerima soal tanpa kunci jawaban, menyimpan jawaban sementara secara terenkripsi di perangkat, lalu mengirim autosave ke edge. Antrean lokal dikosongkan setelah edge memberikan ACK.
- Bila LAN terputus singkat, peserta tetap dapat melanjutkan soal yang sudah diterima. Saat koneksi pulih, antrean jawaban dikirim ulang secara idempoten berdasarkan `answer_revision`.
- Submit final hanya dianggap berhasil setelah edge mengembalikan `submission_receipt`. Jika ACK hilang, client menanyakan status attempt tanpa membuat submit ganda.

2. Kontrak Client ↔ Edge:

- `GET /local/v1/client/bootstrap` — metadata edge, versi protokol, waktu resmi, dan kebijakan sesi.
- `POST /local/v1/client/register` — registrasi instance aplikasi tanpa mengumpulkan fingerprint invasif.
- `POST /local/v1/exams/{id}/attempts` — klaim token dan membuat attempt.
- `GET /local/v1/attempts/{id}/questions` — mengambil soal tanpa kunci jawaban.
- `PUT /local/v1/attempts/{id}/answers/{questionId}` — autosave idempoten dengan nomor revisi.
- `POST /local/v1/attempts/{id}/submit` — final submit dan receipt.
- `GET /local/v1/attempts/{id}/result` — membaca hasil sesuai kebijakan sesi.
- `WS /ws/exam` — sinkronisasi timer, status koneksi, pengumuman operator, dan perintah sesi.

3. Aturan Sistem:

- Nama, merek, source code, protokol, installer, dan siklus rilis client dimiliki proyek SentraUjian.
- Client tidak menyimpan kunci jawaban, pembahasan terkunci, secret edge, atau paket ujian utuh setelah sesi berakhir.
- Cache jawaban lokal dienkripsi dengan kunci per attempt dan dihapus aman setelah receipt final serta masa retensi berakhir.
- Setiap request membawa `protocol_version`, `client_version`, `attempt_id`, `request_id`, dan timestamp agar dapat diaudit dan diduplikasi.
- Mode fullscreen/kiosk pada MVP adalah pengendalian pengalaman pengguna, bukan klaim anti-cheating absolut.
- Update client memakai manifest rilis bertanda tangan. Operator dapat mengunduh installer dari pusat lalu mendistribusikannya ke lab tanpa internet.

12. Alur Navigasi & Arsitektur Layout

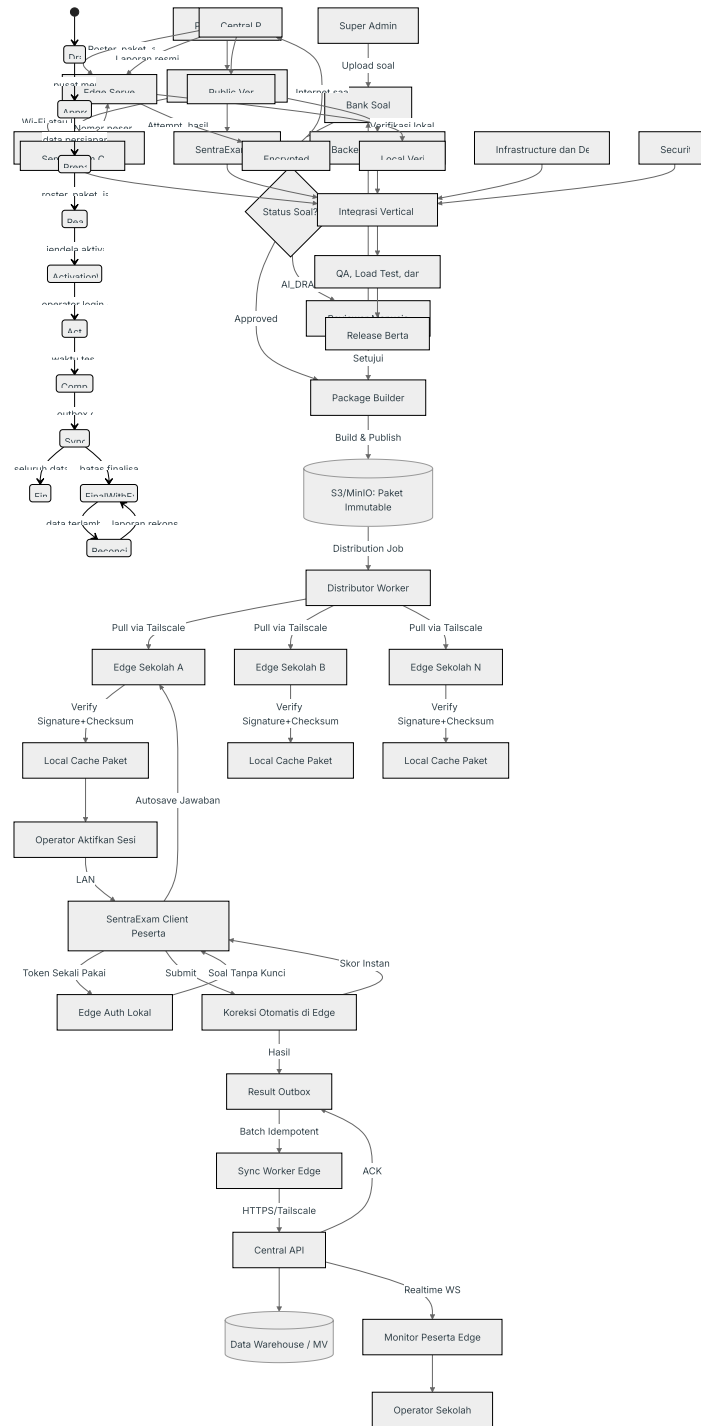
Peta navigasi alur halaman dan struktur tata letak (layout).

Arsitektur Layout (Persisten)

- **Central Public Layout:** Header dengan logo SentraUjian + tombol login + link dokumentasi; Footer berisi kebijakan privasi & status sistem.
- **Central Dashboard Layout:** Sidebar kiri fixed 260px (Overview, Sekolah, Edge, Bank Soal, Review, Package Builder, Distribusi, Jadwal, Hasil, Sinkronisasi Gagal, Audit, ML) + Header kecil berisi breadcrumb, search bar, dan user menu.
- **Edge Admin Layout:** Sidebar kiri 240px (Status Server, LAN, Paket, Jadwal, Peserta, Monitor, Sinkron, Storage, Log, Notifikasi) + badge "LAN 192.168.1.10" pada header.
- **SentraExam Client Layout:** aplikasi desktop fullscreen tanpa sidebar, hanya header timer + badge LAN + tombol
Sebelumnya/Berikutnya, navigasi nomor, penanda ragu-ragu, status autosave, serta dialog konfirmasi submit. Antarmuka tetap nyaman untuk mouse, keyboard, dan layar sentuh.
- **Mobile Operator Layout:** Bottom Tab Navigator (Dashboard, Monitor, Notifikasi, Profil), tanpa drawer.

13. Kebutuhan Non-Fungsional (SEO, Keamanan, & Performa)

- **SEO (Central Web & Halaman Publik):**



- Wajib tag `<title>` dinamis format `[Halaman] – SentraUjian`.
- Meta description unik per halaman, canonical URL, dan `robots.txt` (dashboard `noindex`).
- Open Graph + Twitter Card untuk halaman publik & dokumentasi.
- `sitemap.xml` otomatis untuk halaman `/`, `/docs`, `/privacy`.
- **Edge Web dan Mobile bersifat `noindex` (hanya LAN internal); SentraExam Client adalah aplikasi desktop dan tidak diekspos sebagai situs publik.**

- **Keamanan:**

- **Device identity** per edge: keypair unik (`edge_device_keys`), rotasi 90 hari.
- **Mutual authentication** edge↔pusat via mTLS + HMAC signature pada setiap request.
- **Short-lived access token** JWT (15 menit) + refresh token rotation.
- **Package signature** Ed25519 dan checksum SHA-256 wajib diverifikasi sebelum paket aktif.
- **Envelope encryption:** AES-256-GCM untuk konten, RSA-OAEP untuk wrap DEK.
- **Encryption at rest:** TDE PostgreSQL pusat + full disk encryption di edge.
- **TLS 1.3** wajib untuk semua trafik pusat; edge LAN default HTTP + self-signed HTTPS opsional.
- **Rate limiting** berbasis Redis: 20/menit per IP untuk login, 600/menit per token untuk data.
- **Replay attack protection:** nonce + timestamp ±2 menit pada heartbeat & sync.
- **RBAC** dengan 8 role minimum: `SUPER_ADMIN`, `CONTENT_ADMIN`, `REVIEWER`, `SCHOOL_ADMIN`, `SCHOOL_OPERATOR`, `PROCTOR`, `STUDENT`, `AUDITOR`.
- **Secret** disimpan di Docker secrets / environment manager (bukan source code).
- **Answer key** dan pembahasan terkunci **dilarang** dikirim ke klien peserta.
- **Server time** = sumber waktu resmi seluruh sistem.
- Attempt status `FINAL_SUBMITTED` bersifat **immutable** (kolom `locked_at`).
- Semua perubahan paket dan kunci jawaban tercatat di `audit_logs`.

- **Performa:**

- Optimasi gambar menggunakan WebP/AVIF (`sharp` di worker), lazy loading pada `<Image>`.
- Caching: Redis untuk dataset statis (subjects, topics), browser cache 1 jam untuk assets.
- Readiness check `GET /local/v1/readiness` sebelum sesi ujian diaktifkan (cek paket, storage, DB, waktu).
- Pre-exam readiness validation otomatis 15 menit sebelum jadwal.
- Compress response gzip/brotli, HTTP/2 untuk pusat.
- Target: `/local/v1/attempts/{id}/questions` < 200 ms untuk 60 soal; central dashboard < 1.5 s untuk 1.000 sekolah (dengan pagination).
- Prometheus metrics di semua service; structured logging JSON.
- Autosave jawaban debounced 2 detik untuk menghindari beban tulis berlebih.

14. Panduan Bahasa, Copywriting, & Data Dummy

Panduan nada bicara (Tone of Voice) dan contoh data agar prototipe terasa nyata.

- **Gaya Bahasa:** Profesional, tegas, ramah, dan membumi (menggunakan kata "Anda" untuk pengguna, "Kami" untuk sistem). Hindari jargon berlebihan di UI operator sekolah; gunakan istilah operasional yang mudah dipahami. Hindari kata "bocoran", "prediksi pasti", "cheat", atau "nakal" — ganti dengan "soal latihan berbasis prediksi", "verifikasi", "pengawasan".
- **Instruksi Data Dummy: JANGAN PERNAH MENGGUNAKAN "Lorem Ipsum"**. Gunakan data dummy berbahasa Indonesia yang relevan dan realistis. Contoh spesifik:
 - **Sekolah:** `SMAN 1 Nusantara` (Jakarta), `SMKN 2 Bahari` (Surabaya), `SMA Tunas Bangsa` (Makassar). Kode: `SCH-JKT-001`, `SCH-SBY-002`, `SCH-MKS-003`.
 - **Edge Device:** `EDGE-JKT-001` (Windows Server 2022), `EDGE-SBY-002` (Ubuntu 22.04), `EDGE-MKS-003` (Ubuntu 22.04). IP LAN: `192.168.1.10`, `192.168.2.10`, `192.168.3.10`.

- **Staf Pusat:** `Andi Pratama` (`SUPER_ADMIN`), `Rina Wulandari` (`CONTENT_ADMIN`), `Budi Hartono` (`REVIEWER`), `Siti Aminah` (`AUDITOR`).
- **Operator Sekolah:** `Dewi Lestari` (`SCHOOL_ADMIN SMAN 1 Nusantara`), `Agus Setiawan` (`SCHOOL_OPERATOR SMKN 2 Bahari`).
- **Peserta:** `Ahmad Fauzan` (`NIS 2026001`), `Nabila Putri` (`NIS 2026002`), `Rizky Ramadhan` (`NIS 2026003`).
- **Mata Pelajaran:** `Penalaran Umum (PU)`, `Pengetahuan & Pemahaman Umum (PPU)`, `Pemahaman Bacaan & Menulis (PBM)`, `Pengetahuan Kuantitatif (PK)`, `Literasi Bahasa Indonesia (LBI)`, `Literasi Bahasa Inggris (LBE)`, `Penalaran Matematika (PM)`.
- **Paket Ujian:** `utbk-2026-set-01` dengan `min_edge_version=1.2.0`, durasi 195 menit, 7 subtes, 155 soal.
- **Contoh Soal Dummy:** "Bacalah paragraf berikut. Manakah pernyataan yang paling sesuai dengan simpulan teks di atas? (A) ... (B) ... (C) ... (D) ... (E) ..."
- **Contoh Soal Prediksi:** `AI_DRAFT` dengan `model_name=gpt-exam-v2.1`, `confidence_score=0.83`, `dataset_version=2025-Q4`.
- **Notifikasi Edge Contoh:** "Paket `utbk-2026-set-01` v1.2.0 berhasil diverifikasi dan siap digunakan pada 15 Maret 2026 pukul 07:42 WIB."

15. Fondasi Teknis (Untuk Tim Pengembang / Programmer & AI)

Petunjuk arsitektur teknis spesifik.

- **Bahasa & Framework:**
 - Backend: **Go 1.23+** (Gin/Fiber), `sqlc` + `pgx` untuk query, `zitadel` /custom JWT untuk auth.
 - Frontend Pusat & Edge Admin: **React 19**, **TypeScript strict**, **Vite 6**, **TanStack Router**, **TanStack Query**, **React Hook Form**, **Zod**.
 - Desktop peserta: **Tauri 2 + Rust + React 19 + TypeScript strict + Vite 6**, memakai webview bawaan OS dan installer Windows MSI/NSIS serta Linux Applmage/deb.
 - Mobile Operator: **React Native 0.76+ (Expo SDK 52)**, **TypeScript strict**, **TanStack Query for RN**, **React Hook Form**, **Zod**.
 - Realtime: **WebSocket** (Gorilla/ `nhooyr` di Go, `react-use-websocket` di React).
- **Tampilan Antarmuka (UI):** **Tailwind CSS 4**, **shadcn/ui**, **Lucide Icons**, **Sonner** untuk toast, **Recharts** untuk grafik monitoring.
- **Autentikasi:**
 - Staf pusat: JWT HS512 (access 15 mnt) + Refresh Rotating via Redis.
 - Edge lokal: JWT HS256 (access 30 mnt) + sesi LAN.
 - Peserta: **Token Ujian Sekali Pakai** — random 6 blok karakter (contoh `7F3K-9QM2-NX58-PW01`), hashed SHA-256 di DB, single-use & expiry.
 - Edge↔Pusat: mTLS (Tailscale certs) + HMAC signature.
- **Basis Data (Database):**
 - Pusat: **PostgreSQL 16** + **Redis 7** (cache, job queue via asynq, distributed lock via Redlock, rate limit).
 - Edge: **PostgreSQL 16** (produksi) atau **SQLite WAL** (prototipe); Redis lokal opsional.
 - Storage paket & aset: **S3 / MinIO**.
- **Infrastruktur:** Docker Compose (dev & deployment awal), Tailscale (privat network), Nginx/Caddy (reverse proxy + TLS), Prometheus + Grafana (metrics), structured logging (JSON via `zeroLog`).
- **Monorepo:** **Turborepo** dengan pnpm workspace.
- **ML/DL Pipeline:** Modul terpisah di `central-worker/ml/` (Python service via sidecar), bukan bagian dari critical path ujian.

Struktur Skema Database Nyata (Central PostgreSQL — Drizzle-style TypeScript)

```
// ==== CENTRAL DATABASE (PostgreSQL 16) ====
import { pgTable, uuid, varchar, text, timestamp, boolean, integer, jsonb, bigint, uniqueIndex, index, pgEnum } from
"drizzle-orm/pg-core";

export const questionStatusEnum = pgEnum("question_status",
["DRAFT", "AI_DRAFT", "IN_REVIEW", "APPROVED", "REJECTED", "ARCHIVED"]);

export const distributionStatusEnum = pgEnum("distribution_status",
["PENDING", "DOWNLOADING", "SUCCESS", "VERIFY_FAILED", "VERSION_INCOMPATIBLE", "DEVICE_OFFLINE", "FAILED_PERMANENT"]);

export const roleEnum = pgEnum("role",
["SUPER_ADMIN", "CONTENT_ADMIN", "REVIEWER", "SCHOOL_ADMIN", "SCHOOL_OPERATOR", "PROCTOR", "STUDENT", "AUDITOR"]);

export const organizations = pgTable("organizations", {
  id: uuid("id").primaryKey().defaultRandom(),
  name: varchar("name", { length: 200 }).notNull(),
  type: varchar("type", { length: 40 }).notNull().default("SCHOOL"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const schools = pgTable("schools", {
  id: uuid("id").primaryKey().defaultRandom(),
  organizationId: uuid("organization_id").references(() => organizations.id).notNull(),
  code: varchar("code", { length: 40 }).notNull(),
  name: varchar("name", { length: 200 }).notNull(),
  province: varchar("province", { length: 80 }),
  city: varchar("city", { length: 80 }),
  timezone: varchar("timezone", { length: 40 }).default("Asia/Jakarta").notNull(),
  deletedAt: timestamp("deleted_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqCode: uniqueIndex("schools_code_uniq").on(t.code) }));

export const schoolUsers = pgTable("school_users", {
  id: uuid("id").primaryKey().defaultRandom(),
  schoolId: uuid("school_id").references(() => schools.id).notNull(),
  fullName: varchar("full_name", { length: 200 }).notNull(),
  email: varchar("email", { length: 200 }).notNull(),
  passwordHash: text("password_hash").notNull(),
  role: roleEnum("role").notNull(),
  isActive: boolean("is_active").default(true).notNull(),
  version: integer("version").default(1).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqEmail: uniqueIndex("school_users_email_uniq").on(t.email) }));

export const edgeDevices = pgTable("edge_devices", {
  id: uuid("id").primaryKey().defaultRandom(),
  schoolId: uuid("school_id").references(() => schools.id).notNull(),
  hostname: varchar("hostname", { length: 120 }).notNull(),
  lanAddress: varchar("lan_address", { length: 60 }).notNull(),
  tailscaleName: varchar("tailscale_name", { length: 120 }),
  appVersion: varchar("app_version", { length: 40 }).notNull(),
  status: varchar("status", { length: 40 }).default("OFFLINE").notNull(),
  lastHeartbeatAt: timestamp("last_heartbeat_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const edgeDeviceKeys = pgTable("edge_device_keys", {
  id: uuid("id").primaryKey().defaultRandom(),
  edgeDeviceId: uuid("edge_device_id").references(() => edgeDevices.id).notNull(),
  publicKey: text("public_key").notNull(),
  fingerprint: varchar("fingerprint", { length: 120 }).notNull(),
  isActive: boolean("is_active").default(true).notNull(),
  rotatedAt: timestamp("rotated_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});
```

```

});

export const edgeHeartbeats = pgTable("edge_heartbeats", {
  id: uuid("id").primaryKey().defaultRandom(),
  edgeDeviceId: uuid("edge_device_id").references(() => edgeDevices.id).notNull(),
  appVersion: varchar("app_version", { length: 40 }).notNull(),
  activeParticipants: integer("active_participants").default(0).notNull(),
  storageUsedMb: bigint("storage_used_mb", { mode: "number" }).notNull(),
  metrics: jsonb("metrics"),
  nonce: varchar("nonce", { length: 64 }).notNull(),
  signature: text("signature").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ idxEdge: index("hb_edge_idx").on(t.edgeDeviceId, t.createdAt) }));

export const subjects = pgTable("subjects", {
  id: uuid("id").primaryKey().defaultRandom(),
  code: varchar("code", { length: 20 }).notNull(),
  name: varchar("name", { length: 120 }).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqCode: uniqueIndex("subjects_code_uniq").on(t.code) }));

export const topics = pgTable("topics", {
  id: uuid("id").primaryKey().defaultRandom(),
  subjectId: uuid("subject_id").references(() => subjects.id).notNull(),
  name: varchar("name", { length: 160 }).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const questions = pgTable("questions", {
  id: uuid("id").primaryKey().defaultRandom(),
  subjectId: uuid("subject_id").references(() => subjects.id).notNull(),
  topicId: uuid("topic_id").references(() => topics.id),
  category: varchar("category", { length: 40 }).notNull().default("VERIFIED"), // VERIFIED | PREDICTION
  status: questionStatusEnum("status").notNull().default("DRAFT"),
  questionType: varchar("question_type", { length: 40 }).notNull().default("TEXT"), // TEXT | IMAGE_QUESTION
  content: text("content").notNull(),
  difficulty: varchar("difficulty", { length: 20 }).default("MEDIUM"),
  answerKeyEnc: text("answer_key_enc").notNull(), // AES-GCM ciphertext
  explanationEnc: text("explanation_enc"), // AES-GCM ciphertext (locked)
  metadata: jsonb("metadata"),
  version: integer("version").default(1).notNull(),
  deletedAt: timestamp("deleted_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const questionOptions = pgTable("question_options", {
  id: uuid("id").primaryKey().defaultRandom(),
  questionId: uuid("question_id").references(() => questions.id).notNull(),
  label: varchar("label", { length: 4 }).notNull(), // A/B/C/D/E
  content: text("content").notNull(),
  optionOrder: integer("option_order").notNull(),
});

export const questionAssets = pgTable("question_assets", {
  id: uuid("id").primaryKey().defaultRandom(),
  questionId: uuid("question_id").references(() => questions.id).notNull(),
  objectKey: varchar("object_key", { length: 300 }).notNull(),
  contentHash: varchar("content_hash", { length: 128 }).notNull(),
  mimeType: varchar("mime_type", { length: 60 }).notNull(),
  sizeBytes: bigint("size_bytes", { mode: "number" }).notNull(),
  originalObjectKey: varchar("original_object_key", { length: 300 }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqHash: uniqueIndex("qa_hash_uniq").on(t.contentHash) }));

export const questionReviews = pgTable("question_reviews", {
  id: uuid("id").primaryKey().defaultRandom(),

```

```

questionId: uuid("question_id").references(() => questions.id).notNull(),
reviewerId: uuid("reviewer_id").references(() => schoolUsers.id).notNull(),
decision: varchar("decision", { length: 20 }).notNull(), // APPROVED | REJECTED | REVISE
reason: text("reason"),
createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const predictionRuns = pgTable("prediction_runs", {
  id: uuid("id").primaryKey().defaultRandom(),
  modelName: varchar("model_name", { length: 120 }).notNull(),
  modelVersion: varchar("model_version", { length: 40 }).notNull(),
  datasetVersion: varchar("dataset_version", { length: 40 }).notNull(),
  generatedQuestionId: uuid("generated_question_id").references(() => questions.id),
  confidenceScore: integer("confidence_score").notNull(), // 0..10000 (x10000)
  generatedAt: timestamp("generated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const examPackages = pgTable("exam_packages", {
  id: uuid("id").primaryKey().defaultRandom(),
  code: varchar("code", { length: 120 }).notNull(), // utbk-2026-set-01
  name: varchar("name", { length: 200 }).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqCode: uniqueIndex("pkg_code_uniq").on(t.code) }));

export const examPackageVersions = pgTable("exam_package_versions", {
  id: uuid("id").primaryKey().defaultRandom(),
  examPackageId: uuid("exam_package_id").references(() => examPackages.id).notNull(),
  version: varchar("version", { length: 20 }).notNull(), // v1, v2
  status: varchar("status", { length: 20 }).notNull().default("DRAFT"), // DRAFT | PUBLISHED | ARCHIVED
  minEdgeVersion: varchar("min_edge_version", { length: 40 }).notNull(),
  durationMinutes: integer("duration_minutes").notNull(),
  manifestJson: jsonb("manifest_json").notNull(),
  s3Prefix: varchar("s3_prefix", { length: 300 }).notNull(),
  signatureEd25519: text("signature_ed25519").notNull(),
  checksumsJson: jsonb("checksums_json").notNull(),
  publishedAt: timestamp("published_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqVer: uniqueIndex("pkg_ver_uniq").on(t.examPackageId, t.version) }));

export const examPackageItems = pgTable("exam_package_items", {
  id: uuid("id").primaryKey().defaultRandom(),
  examPackageVersionId: uuid("exam_package_version_id").references(() => examPackageVersions.id).notNull(),
  questionId: uuid("question_id").references(() => questions.id).notNull(),
  subtestCode: varchar("subtest_code", { length: 20 }).notNull(),
  itemOrder: integer("item_order").notNull(),
});

export const packageDistributions = pgTable("package_distributions", {
  id: uuid("id").primaryKey().defaultRandom(),
  examPackageVersionId: uuid("exam_package_version_id").references(() => examPackageVersions.id).notNull(),
  schoolId: uuid("school_id").references(() => schools.id).notNull(),
  edgeDeviceId: uuid("edge_device_id").references(() => edgeDevices.id).notNull(),
  status: distributionStatusEnum("status").notNull().default("PENDING"),
  attempts: integer("attempts").default(0).notNull(),
  acknowledgedAt: timestamp("acknowledged_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const distributionEvents = pgTable("distribution_events", {
  id: uuid("id").primaryKey().defaultRandom(),
  packageDistributionId: uuid("package_distribution_id").references(() => packageDistributions.id).notNull(),
  eventType: varchar("event_type", { length: 40 }).notNull(),
  payload: jsonb("payload"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const examSchedules = pgTable("exam_schedules", {

```

```

    id: uuid("id").primaryKey().defaultRandom(),
    examPackageVersionId: uuid("exam_package_version_id").references(() => examPackageVersions.id).notNull(),
    schoolId: uuid("school_id").references(() => schools.id).notNull(),
    startsAt: timestamp("starts_at", { withTimezone: true }).notNull(),
    endsAt: timestamp("ends_at", { withTimezone: true }).notNull(),
    createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  });

export const centralAttempts = pgTable("central_attempts", {
  id: uuid("id").primaryKey().defaultRandom(),
  globalUuid: varchar("global_uuid", { length: 40 }).notNull(), // ULID dari edge
  idempotencyKey: varchar("idempotency_key", { length: 80 }).notNull(),
  schoolId: uuid("school_id").references(() => schools.id).notNull(),
  examPackageVersionId: uuid("exam_package_version_id").references(() => examPackageVersions.id).notNull(),
  studentRef: varchar("student_ref", { length: 120 }).notNull(),
  score: integer("score"),
  rawScore: jsonb("raw_score"),
  submittedAt: timestamp("submitted_at", { withTimezone: true }).notNull(),
  syncedAt: timestamp("synced_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqIdem: uniqueIndex("attempts_idem_uniq").on(t.idempotencyKey), uniqUuid:
uniqueIndex("attempts_uuid_uniq").on(t.globalUuid) }));

export const centralAttemptAnswers = pgTable("central_attempt_answers", {
  id: uuid("id").primaryKey().defaultRandom(),
  centralAttemptId: uuid("central_attempt_id").references(() => centralAttempts.id).notNull(),
  questionId: uuid("question_id").references(() => questions.id).notNull(),
  studentResponse: varchar("student_response", { length: 80 }).notNull(),
  isCorrect: boolean("is_correct").notNull(),
});

export const centralResults = pgTable("central_results", {
  id: uuid("id").primaryKey().defaultRandom(),
  centralAttemptId: uuid("central_attempt_id").references(() => centralAttempts.id).notNull(),
  totalScore: integer("total_score").notNull(),
  subtestScores: jsonb("subtest_scores").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const syncBatches = pgTable("sync_batches", {
  id: uuid("id").primaryKey().defaultRandom(),
  edgeDeviceId: uuid("edge_device_id").references(() => edgeDevices.id).notNull(),
  batchId: varchar("batch_id", { length: 40 }).notNull(),
  status: varchar("status", { length: 30 }).notNull().default("RECEIVED"),
  attemptCount: integer("attempt_count").notNull(),
  receivedAt: timestamp("received_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqBatch: uniqueIndex("sync_batch_uniq").on(t.edgeDeviceId, t.batchId) }));

export const syncEvents = pgTable("sync_events", {
  id: uuid("id").primaryKey().defaultRandom(),
  syncBatchId: uuid("sync_batch_id").references(() => syncBatches.id).notNull(),
  eventType: varchar("event_type", { length: 40 }).notNull(),
  payload: jsonb("payload"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const auditLogs = pgTable("audit_logs", {
  id: uuid("id").primaryKey().defaultRandom(),
  actorId: uuid("actor_id"),
  actorRole: varchar("actor_role", { length: 40 }),
  action: varchar("action", { length: 80 }).notNull(),
  entity: varchar("entity", { length: 80 }).notNull(),
  entityId: varchar("entity_id", { length: 80 }),
  payload: jsonb("payload"),
  ipAddress: varchar("ip_address", { length: 60 }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ idxActor: index("audit_actor_idx").on(t.actorId, t.createdAt) }));

```

Struktur Skema Database Nyata (Edge PostgreSQL — Drizzle-style TypeScript)

```
// ==== EDGE DATABASE (PostgreSQL 16 / SQLite WAL kompatibel) ====
import { pgTable, uuid, varchar, text, timestamp, boolean, integer, jsonb, bigint, uniqueIndex, index } from "drizzle-orm/pg-core";

export const edgeConfiguration = pgTable("edge_configuration", {
  id: uuid("id").primaryKey().defaultRandom(),
  schoolCode: varchar("school_code", { length: 40 }).notNull(),
  schoolName: varchar("school_name", { length: 200 }).notNull(),
  lanAddress: varchar("lan_address", { length: 60 }).notNull(),
  centralBaseUrl: varchar("central_base_url", { length: 200 }).notNull(),
  tokenTtlMinutes: integer("token_ttl_minutes").default(60).notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const localUsers = pgTable("local_users", {
  id: uuid("id").primaryKey().defaultRandom(),
  fullName: varchar("full_name", { length: 200 }).notNull(),
  username: varchar("username", { length: 120 }).notNull(),
  passwordHash: text("password_hash").notNull(),
  role: varchar("role", { length: 40 }).notNull(),
  isActive: boolean("is_active").default(true).notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqUser: uniqueIndex("local_users_username_uniq").on(t.username) }));

export const packageCache = pgTable("package_cache", {
  id: uuid("id").primaryKey().defaultRandom(),
  packageCode: varchar("package_code", { length: 120 }).notNull(),
  version: varchar("version", { length: 20 }).notNull(),
  minEdgeVersion: varchar("min_edge_version", { length: 40 }).notNull(),
  verified: boolean("verified").default(false).notNull(),
  signatureOk: boolean("signature_ok").default(false).notNull(),
  checksumOk: boolean("checksum_ok").default(false).notNull(),
  sizeBytes: bigint("size_bytes", { mode: "number" }).notNull(),
  cachedAt: timestamp("cached_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqPkg: uniqueIndex("pkg_cache_uniq").on(t.packageCode, t.version) }));

export const packageFiles = pgTable("package_files", {
  id: uuid("id").primaryKey().defaultRandom(),
  packageCacheId: uuid("package_cache_id").references(() => packageCache.id).notNull(),
  relativePath: varchar("relative_path", { length: 300 }).notNull(),
  sha256: varchar("sha256", { length: 128 }).notNull(),
  sizeBytes: bigint("size_bytes", { mode: "number" }).notNull(),
});

export const localExamSessions = pgTable("local_exam_sessions", {
  id: uuid("id").primaryKey().defaultRandom(),
  packageCacheId: uuid("package_cache_id").references(() => packageCache.id).notNull(),
  subtestCode: varchar("subtest_code", { length: 20 }).notNull(),
  status: varchar("status", { length: 20 }).default("SCHEDULED").notNull(), // SCHEDULED | ACTIVE | FINISHED
  startsAt: timestamp("starts_at", { withTimezone: true }).notNull(),
  endsAt: timestamp("ends_at", { withTimezone: true }).notNull(),
  activatedByUserId: uuid("activated_by_user_id").references(() => localUsers.id),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const localAttempts = pgTable("local_attempts", {
  id: uuid("id").primaryKey().defaultRandom(),
  globalUuid: varchar("global_uuid", { length: 40 }).notNull(),
  localExamSessionId: uuid("local_exam_session_id").references(() => localExamSessions.id).notNull(),
  participantTokenHash: varchar("participant_token_hash", { length: 128 }).notNull(),
  participantName: varchar("participant_name", { length: 200 }).notNull(),
  participantNis: varchar("participant_nis", { length: 40 }).notNull(),
  clientInstanceId: uuid("client_instance_id"),
  status: varchar("status", { length: 20 }).default("STARTED").notNull(), // STARTED | SUBMITTED | LOCKED
```

```

    startedAt: timestamp("started_at", { withTimezone: true }).defaultNow().notNull(),
    lockedAt: timestamp("locked_at", { withTimezone: true }),
    serverDeadline: timestamp("server_deadline", { withTimezone: true }).notNull(),
  }, (t) => ({ uniqUuid: uniqueIndex("local_attempt_uuid_uniq").on(t.globalUuid) }));

export const localAttemptAnswers = pgTable("local_attempt_answers", {
  id: uuid("id").primaryKey().defaultRandom(),
  localAttemptId: uuid("local_attempt_id").references(() => localAttempts.id).notNull(),
  questionId: varchar("question_id", { length: 40 }).notNull(),
  studentResponse: varchar("student_response", { length: 80 }),
  revision: integer("revision").default(1).notNull(),
  requestId: varchar("request_id", { length: 40 }).notNull(),
  isCorrect: boolean("is_correct"),
  answeredAt: timestamp("answered_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({
  idxAttempt: index("laa_attempt_idx").on(t.localAttemptId, t.questionId),
  uniqRequest: uniqueIndex("laa_request_uniq").on(t.localAttemptId, t.requestId),
}));

export const localResults = pgTable("local_results", {
  id: uuid("id").primaryKey().defaultRandom(),
  localAttemptId: uuid("local_attempt_id").references(() => localAttempts.id).notNull(),
  totalScore: integer("total_score").notNull(),
  subtestScores: jsonb("subtest_scores").notNull(),
  computedAt: timestamp("computed_at", { withTimezone: true }).defaultNow().notNull(),
});

export const resultOutbox = pgTable("result_outbox", {
  id: uuid("id").primaryKey().defaultRandom(),
  localResultId: uuid("local_result_id").references(() => localResults.id).notNull(),
  idempotencyKey: varchar("idempotency_key", { length: 80 }).notNull(),
  status: varchar("status", { length: 20 }).default("PENDING").notNull(), // PENDING | SENDING | SYNCED | FAILED
  attempts: integer("attempts").default(0).notNull(),
  lastError: text("last_error"),
  nextRetryAt: timestamp("next_retry_at", { withTimezone: true }),
  syncedAt: timestamp("synced_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqIdem: uniqueIndex("outbox_idem_uniq").on(t.idempotencyKey) }));

export const syncCheckpoints = pgTable("sync_checkpoints", {
  id: uuid("id").primaryKey().defaultRandom(),
  batchId: varchar("batch_id", { length: 40 }).notNull(),
  lastSyncedOutboxId: uuid("last_synced_outbox_id"),
  syncedAt: timestamp("synced_at", { withTimezone: true }).defaultNow().notNull(),
});

export const edgeAuditLogs = pgTable("edge_audit_logs", {
  id: uuid("id").primaryKey().defaultRandom(),
  actorRole: varchar("actor_role", { length: 40 }),
  action: varchar("action", { length: 80 }).notNull(),
  entity: varchar("entity", { length: 80 }).notNull(),
  entityId: varchar("entity_id", { length: 80 }),
  payload: jsonb("payload"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const edgeNotifications = pgTable("edge_notifications", {
  id: uuid("id").primaryKey().defaultRandom(),
  level: varchar("level", { length: 20 }).notNull(), // info | success | warning | danger
  title: varchar("title", { length: 200 }).notNull(),
  message: text("message").notNull(),
  isRead: boolean("is_read").default(false).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const clientInstances = pgTable("client_instances", {
  id: uuid("id").primaryKey().defaultRandom(),

```

```

installationId: varchar("installation_id", { length: 80 }).notNull(),
deviceLabel: varchar("device_label", { length: 120 }),
clientVersion: varchar("client_version", { length: 40 }).notNull(),
protocolVersion: varchar("protocol_version", { length: 20 }).notNull(),
platform: varchar("platform", { length: 30 }).notNull(), // windows | linux
lastSeenAt: timestamp("last_seen_at", { withTimezone: true }).defaultNow().notNull(),
createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqInstall: uniqueIndex("client_installation_uniq").on(t.installationId) }));

export const submissionReceipts = pgTable("submission_receipts", {
  id: uuid("id").primaryKey().defaultRandom(),
  localAttemptId: uuid("local_attempt_id").references(() => localAttempts.id).notNull(),
  receiptCode: varchar("receipt_code", { length: 80 }).notNull(),
  answerDigest: varchar("answer_digest", { length: 128 }).notNull(),
  submittedAt: timestamp("submitted_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({
  uniqAttempt: uniqueIndex("submission_receipt_attempt_uniq").on(t.localAttemptId),
  uniqReceipt: uniqueIndex("submission_receipt_code_uniq").on(t.receiptCode),
}));

```

Struktur Tabel Tambahan (Peserta & Token Sekali Pakai)

```

// Tambahan edge DB: peserta & token ujian sekali pakai
export const localParticipants = pgTable("local_participants", {
  id: uuid("id").primaryKey().defaultRandom(),
  nis: varchar("nis", { length: 40 }).notNull(),
  fullName: varchar("full_name", { length: 200 }).notNull(),
  className: varchar("class_name", { length: 40 }).notNull(),
  examSessionId: uuid("exam_session_id").references(() => localExamSessions.id).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqNis: uniqueIndex("local_participant_nis_uniq").on(t.nis, t.examSessionId) }));

export const examTokens = pgTable("exam_tokens", {
  id: uuid("id").primaryKey().defaultRandom(),
  participantId: uuid("participant_id").references(() => localParticipants.id).notNull(),
  tokenHash: varchar("token_hash", { length: 128 }).notNull(),
  expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
  claimedAt: timestamp("claimed_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({ uniqHash: uniqueIndex("exam_tokens_hash_uniq").on(t.tokenHash) }));

```

Variabel Lingkungan (`.env.example`)

```
# ===== CENTRAL API =====
NEXT_PUBLIC_APP_URL=http://localhost:3000
NODE_ENV=development
GO_ENV=development
CENTRAL_HTTP_PORT=8080
CENTRAL_HTTP_HOST=0.0.0.0
CENTRAL_BASE_URL=http://localhost:8080

# --- Database (PostgreSQL Pusat) ---
DATABASE_URL=postgres://sentra:secret@central-postgres:5432/sentraujian_central
DATABASE_MAX_OPEN_CONNS=50
DATABASE_MAX_IDLE_CONNS=10

# --- Redis ---
REDIS_URL=redis://central-redis:6379
REDIS_PREFIX=sentra:

# --- S3 / MinIO ---
S3_ENDPOINT=http://central-minio:9000
S3_REGION=us-east-1
S3_BUCKET_PACKAGES=sentraujian-packages
S3_BUCKET_ASSETS=sentraujian-assets
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_USE_SSL=false

# --- JWT & Crypto ---
JWT_ACCESS_SECRET=replace-with-strong-random-secret
JWT_REFRESH_SECRET=replace-with-strong-random-secret
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=168h
PACKAGE_SIGNING_PRIVATE_KEY_PATH=/run/secrets/package_signing_private.pem
PACKAGE_SIGNING_PUBLIC_KEY_PATH=/run/secrets/package_signing_public.pem
ENVELOPE_DEK_ROTATION_DAYS=30

# --- Observability ---
PROMETHEUS_METRICS_ENABLED=true
LOG_LEVEL=info
LOG_FORMAT=json
OTEL_EXPORTER_OTLP_ENDPOINT=

# ===== EDGE SERVER =====
EDGE_HTTP_PORT=80
EDGE_HTTP_HOST=0.0.0.0
EDGE_LAN_ADDRESS=http://192.168.1.10
EDGE_SCHOOL_CODE=SCH-JKT-001
EDGE_SCHOOL_NAME="SMAN 1 Nusantara"
EDGE_CENTRAL_BASE_URL=http://central.tailnet.ts.net:8080
EDGE_DEVICE_ID=EDGE-JKT-001
EDGE_DEVICE_SECRET=replace-with-unique-random-secret
EDGE_DEVICE_PRIVATE_KEY_PATH=/run/secrets/edge_private.pem
EDGE_DEVICE_PUBLIC_KEY_PATH=/run/secrets/edge_public.pem

# --- Database Edge (PostgreSQL / SQLite WAL) ---
EDGE_DB_ENGINE=postgres
EDGE_DATABASE_URL=postgres://edge:secret@edge-postgres:5432/sentraujian_edge
EDGE_SQLITE_PATH=/var/lib/sentraujian/edge.db

# --- Local storage & cache ---
EDGE_CACHE_DIR=/var/lib/sentraujian/packages
EDGE_STORAGE_WARN_PCT=80
EDGE_STORAGE_CRITICAL_PCT=95

# --- Token Ujian Sekali Pakai ---
```

```
EDGE_EXAM_TOKEN_TTL_MINUTES=60
EDGE_EXAM_TOKEN_LENGTH=16
EDGE_EXAM_TOKEN_PREFIX=SNT

# --- WebSocket (LAN) ---
EDGE_WS_PORT=8081
EDGE_WS_PATH=/ws/monitor
EDGE_EXAM_WS_PATH=/ws/exam

# ===== FRONTEND (Central & Edge Admin) =====
VITE_CENTRAL_API_BASE_URL=http://localhost:8080
VITE_EDGE_API_BASE_URL=http://192.168.1.10
VITE_WS_MONITOR_URL=ws://192.168.1.10:8081/ws/monitor

# ===== SENTRAEXAM CLIENT (Tauri Desktop) =====
VITE_EXAM_EDGE_BASE_URL=http://ujian.local
VITE_EXAM_WS_URL=ws://ujian.local:8081/ws/exam
EXAM_PROTOCOL_VERSION=1
EXAM_CLIENT_CHANNEL=stable
EXAM_LOCAL_CACHE_TTL_HOURS=24
TAURI_SIGNING_PUBLIC_KEY=replace-with-release-signing-public-key
TAURI_UPDATE_MANIFEST_URL=http://ujian.local/client-updates/stable.json

# ===== MOBILE (React Native) =====
EXO_PUBLIC_EDGE_BASE_URL=http://192.168.1.10
EXO_PUBLIC_WS_MONITOR_URL=ws://192.168.1.10:8081/ws/monitor

# ===== TAILSCALE (Operasional, bukan trafik ujian) =====
TAILSCALE_AUTH_KEY=tskey-auth-replace-me
TAILSCALE_HOSTNAME=sentra-central-01

# ===== DOCKER COMPOSE (dev) =====
COMPOSE_PROJECT_NAME=sentraujian
POSTGRES_USER=sentra
POSTGRES_PASSWORD=secret_pgsql
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=minioadmin
```

16. Struktur Tim, Ownership, dan Pola Kolaborasi

Pembagian ini membuat beberapa tim dapat bekerja paralel dengan batas tanggung jawab, kontrak, dan hasil kerja yang jelas.

A. Susunan Tim

Tim	Peran Inti	Ownership Utama	Deliverable Wajib
Product & Business Analyst	Product Owner, Business Analyst, Scrum Master	ruang lingkup, prioritas, alur bisnis, acceptance criteria	PRD, backlog, decision log, release scope, UAT scenario
UI/UX & Design System	Product Designer, UX Researcher, Design System Designer	pengalaman admin pusat, operator sekolah, pengawas, dan peserta	user flow, wireframe, prototype, design tokens, accessibility spec
Central Frontend	React Engineer	apps/central-web , dashboard pusat, bank soal, distribusi, rekap	halaman pusat, state/query, RBAC route guard, frontend tests
Edge Frontend & Mobile	React Engineer, React Native Engineer	apps/edge-admin-web , apps/edge-operator-mobile	dashboard LAN, monitoring realtime, notifikasi, mobile operator
SentraExam Client	Tauri/Rust Engineer, React Engineer	apps/exam-client , installer dan lifecycle aplikasi desktop peserta	launcher, system check, exam runner, recovery, signed installer

Tim	Peran Inti	Ownership Utama	Deliverable Wajib
Backend Central	Go Engineer, Data Engineer	apps/central-api , apps/central-worker , database pusat	API, auth/RBAC, package builder, distribusi, rekap, audit
Backend Edge	Go Engineer, Database Engineer	apps/edge-server , database lokal, LAN API, scoring, outbox	edge binary, exam session, token, autosave, scoring, sync
Infrastructure & DevOps	DevOps/SRE, Network Engineer	VPS pusat, server sekolah, Tailscale, DNS/LAN, CI/CD, observability	IaC, Docker, installer edge, monitoring, backup, runbook
Security	AppSec Engineer, Security Reviewer	threat model, key management, signing, hardening, audit	security baseline, review checklist, pentest report, incident plan
QA & Performance	QA Engineer, Automation Engineer, Performance Engineer	test plan lintas aplikasi dan ketahanan 1.000 sekolah	unit/integration/E2E, compatibility matrix, k6 report, release sign-off
Content & Operations	Content Admin Lead, Training/Support	bank soal, SOP hari-H, onboarding sekolah, dukungan	template import, SOP operator, materi pelatihan, support escalation

B. Batas Ownership Teknis

- **Central Frontend** tidak mengakses database secara langsung; seluruh data melalui kontrak OpenAPI Central API.
- **Edge Frontend, Mobile, dan SentraExam Client** hanya mengakses API/WS Edge di LAN dan tidak mengakses PostgreSQL lokal secara langsung.
- **SentraExam Client** tidak berkomunikasi langsung dengan server pusat saat ujian. Semua trafik peserta berhenti di edge sekolah.
- **Backend Central** memiliki skema data pusat dan kontrak distribusi/sinkronisasi; **Backend Edge** memiliki skema lokal, lifecycle attempt, scoring, dan outbox.
- **Infrastruktur** memiliki deployment, jaringan, sertifikat, secret injection, backup, observability, dan rollback; tim aplikasi tetap memiliki health/readiness endpoint.
- **Security** menetapkan kontrol wajib dan menjadi reviewer perubahan auth, kriptografi, package signing, token, audit, dan release client.
- **QA** memiliki test plan serta bukti kelulusan, tetapi perbaikan defect tetap dimiliki tim pembuat komponen.
- Perubahan kontrak lintas tim wajib melalui proposal singkat, update OpenAPI/schema, compatibility note, dan persetujuan kedua pemilik komponen.

C. Matriks RACI Ringkas

Keterangan: **R** = mengerjakan, **A** = penanggung jawab akhir, **C** = dikonsultasikan, **I** = diberi informasi.

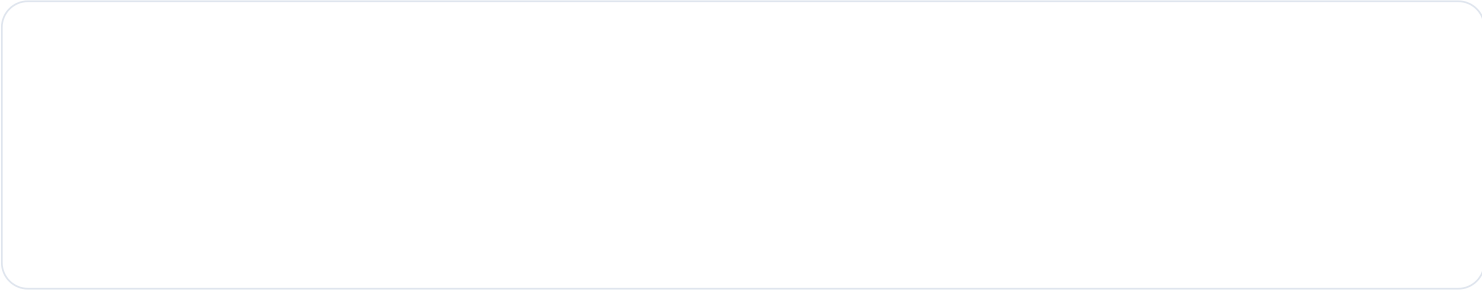
Area	Product	UI/UX	Central FE	Edge FE/Mobile	Exam Client	Backend Central	Backend Edge	Infra	Security	QA
Alur bisnis & scope	A/R	C	I	I	I	C	C	I	C	C
Design system	C	A/R	R	R	R	I	I	I	C	C
Bank soal & dashboard pusat	C	C	A/R	I	I	R	I	C	C	R
Distribusi paket ke sekolah	C	I	R	R	I	A/R	R	R	C	R
Sesi ujian lokal & scoring	C	C	I	R	R	C	A/R	C	C	R
SentraExam Client	C	R	I	I	A/R	I	R	C	C	R
Sinkronisasi hasil	C	I	R	R	I	A/R	R	R	C	R
Jaringan, deployment, monitoring	I	I	C	C	C	C	C	A/R	C	R
Security review & signing	I	I	C	C	R	R	R	R	A	R
Release acceptance	A	C	C	C	C	C	C	C	C	R

D. Kontrak Antar-Tim

1. **Contract-first:** Backend Central dan Backend Edge mempublikasikan OpenAPI 3.1, JSON Schema event, serta contoh payload sebelum frontend mengikat data nyata.

- 2. **Versioning**: kontrak Central API memakai `/api/v1` ; protokol client-edge memakai `protocol_version` . Perubahan breaking membuat versi baru, bukan mengubah diam-diam.
- 3. **Mock resmi**: setiap pemilik API menyediakan mock server dan fixtures Indonesia yang sama agar frontend, mobile, client, dan QA dapat bekerja paralel.
- 4. **Definition of Done**: kode, test, dokumentasi, telemetry, migration/rollback, security check, dan acceptance criteria harus selesai sebelum task dianggap done.
- 5. **Branch dan review**: satu pull request berfokus pada satu task; minimal satu reviewer dari tim pemilik dan satu reviewer dari tim konsumen untuk perubahan lintas kontrak.
- 6. **Decision log**: keputusan arsitektur penting dicatat sebagai ADR di `docs/adr/ADR-XXXX-judul.md` .
- 7. **Integrasi rutin**: build utama dan E2E vertical slice dijalankan setiap hari agar integrasi antartim tidak menumpuk menjelang rilis.

E. Jalur Dependensi Pekerjaan



F. Ritme Eksekusi dan Serah Terima

- Sprint 2 minggu dengan planning, contract review, demo, retrospective, dan release readiness review.
- Daily sync lintas lead maksimal 15 menit khusus blocker dan perubahan kontrak.
- Setiap tim menjaga papan backlog sendiri, tetapi milestone dan dependency ditampilkan pada satu program board.
- Handoff wajib menyertakan tautan PR, versi kontrak, cara menjalankan, data uji, hasil test, risiko tersisa, dan langkah rollback.
- Release dimulai dari lab internal → 3 sekolah pilot → 25 sekolah → 100 sekolah → rollout bertahap sampai 1.000 sekolah berdasarkan error budget.

17. Tahapan Pengerjaan & Task Breakdown (Actionable Work Breakdown Structure)

Daftar tugas terstruktur dan terurut (Atomic Tasks) dengan format checklist markdown `- [ ] **Task X.Y**` . Dirancang khusus agar pengguna dapat menginstruksikan AI Coding Assistant (Antigravity, Cursor, Claude Code, Roo Code, dll.) untuk mengeksekusi proyek langkah demi langkah secara terukur, modular, dan bebas dari kehabisan context window.

MODE EKSEKUSI: PHASE — Setiap Fase harus diselesaikan penuh dalam satu putaran, lalu AI berhenti melapor dan menunggu izin user sebelum masuk fase berikutnya.

Tahap 1: Fondasi Monorepo, Kontrak, UI/UX Dummy (Central Web, Edge Admin Web, Mobile, SentraExam Client)

Tujuan: Membangun seluruh antarmuka visual dan kontrak API/Lib TypeScript secara 100% lengkap dengan data dummy sebelum menyentuh database & Go backend.

- ❑ **Task 1.1 (Monorepo Bootstrap & Design System) — Owner: UI/UX + Frontend Platform:** Setup Turborepo + pnpm workspace sesuai blueprint (`apps/*` , `packages/*`). Konfigurasi TypeScript strict, ESLint, Prettier, Vite, Tailwind CSS 4, shadcn/ui base install (Button, Card, Input, Dialog, Table, Badge, Dropdown, Tabs, Sheet, Sonner, Progress, Skeleton). Definisikan token warna/font dari Bab 4 (Primary `HSL(215,82%,42%)` , font Geist + Inter + JetBrains Mono).
- ❑ **Task 1.2 (Shared Packages):** Buat `packages/api-contracts` (type TypeScript Zod untuk semua entitas), `packages/validation` (schema Zod soal, paket, token, distribusi), `packages/exam-package` (parser manifest & checksums), `packages/ui` (design system), `packages/shared-config` (konstanta env, role, status enum).
- ❑ **Task 1.3 (Central Web — Layout & Routing):** Setup TanStack Router di `apps/central-web` ; buat root layout + Central Dashboard Layout (sidebar 260px + header), buat seluruh halaman publik Bab 3.B (`/login` , `/forgot-password` , `/health`) dengan form React Hook Form + Zod — semua memakai **dummy data typed**.
- ❑ **Task 1.4 (Central Web — Super Admin Pages):** Buat seluruh halaman `/dashboard` , `/schools` , `/schools/new` , `/schools/:id` , `/edges` , `/edges/:id` , `/distributions` , `/schedules` , `/results` , `/sync-failures` , `/audit-log` , `/settings/ml` , `/profile` dengan komponen tabel, filter, kartu status, chart Recharts — data dummy SMAN 1 Nusantara, SMKN 2 Bahari, SMA Tunas Bangsa.
- ❑ **Task 1.5 (Central Web — Content, Review, Package Builder):** Buat halaman `/question-bank` , `/question-bank/prediction` , `/question-bank/import` , `/question-bank/:id` , `/reviews` , `/package-builder` , `/package-builder/new` (wizard 4 langkah), `/package-builder/:id` — memakai data dummy soal `APPROVED` dan `AI_DRAFT` .
- ❑ **Task 1.6 (Edge Admin Web — Layout & Routing):** Setup TanStack Router di `apps/edge-admin-web` ; buat Edge Admin Layout (sidebar 240px, badge LAN `192.168.1.10` , panel In-App Notification Edge via Sonner + halaman `/local/notifications`).
- ❑ **Task 1.7 (Edge Admin Web — Halaman Operator):** Buat seluruh halaman `/local/dashboard` , `/local/network` , `/local/packages` , `/local/schedules` (tombol **Aktifkan Sesi** + modal konfirmasi), `/local/participants` (generate + cetak **Token Ujian Sekali Pakai** format `SNT-XXXX-XXXX-XXXX`), `/local/monitor` (grid WebSocket dummy realtime: peserta aktif/selesai/terputus), `/local/sync` , `/local/storage` , `/local/logs` , `/local/settings` .
- ❑ **Task 1.8 (React Native — Mobile Operator):** Setup `apps/edge-operator-mobile` (Expo SDK 52 + TanStack Router/Expo Router + TanStack Query). Buat halaman `/mobile/login` , `/mobile/dashboard` , `/mobile/monitor` , `/mobile/notifications` dengan tab navigator & styling dari Bab 4.
- ❑ **Task 1.9 (SentraExam Client — Desktop Peserta) — Owner: Tim SentraExam Client:** Setup `apps/exam-client` memakai Tauri 2 + Rust + React + Vite. Buat launcher, `/client/system-check` , `/client/login` , `/client/exam` , `/client/reconnect` , `/client/result` , mode fullscreen, keyboard shortcuts, navigasi nomor, timer resmi, autosave indicator, antrean jawaban lokal terenkripsi, serta skeleton installer Windows/Linux. Pada tahap ini gunakan mock server resmi dan state dummy; aplikasinya tetap produk SentraUjian sendiri, bukan adapter vendor.
- ❑ **Task 1.10 (Storybook & Dokumentasi UI):** Tambahkan Storybook untuk `packages/ui` (minimal 20 komponen terdokumentasi) + `docs/architecture/README.md` awal.

Tahap 2: Backend Go, Database, Auth, Distribusi, Edge MVP, Sync Idempotent

Tujuan: Menghidupkan aplikasi dengan PostgreSQL, Redis, S3/MinIO, Go API, edge server, dan WebSocket monitor realtime.

- ❑ **Task 2.1 (Central API — Bootstrap):** Setup `apps/central-api` Go + Gin + pgx + sqlc. Konfigurasi `zerolog` , `prom-client` , health/readiness endpoint, Redis client, S3 client, Drizzle-like schema menjadi migration SQL (`schema.ts` → `migrations/*.sql`) sesuai Bab 10.
- ❑ **Task 2.2 (Central API — Auth & RBAC):** Implementasi `POST /api/v1/auth/login` , JWT HS512 access 15m + refresh 7d (Redis), middleware RBAC 8 role (`SUPER_ADMIN` , `CONTENT_ADMIN` , `REVIEWER` , `SCHOOL_ADMIN` , `SCHOOL_OPERATOR` , `PROCTOR` , `STUDENT` , `AUDITOR`), rate limiter Redis (20/menit login).
- ❑ **Task 2.3 (Central API — Questions & Reviews):** Implementasi endpoint Bab 6: `POST /questions/import` , `POST /questions` , `PATCH /questions/{id}` , `POST /questions/{id}/review` . Parser JSON/CSV/XLSX/PDF/DOCX + validasi Zod-Go, upload aset ke MinIO dengan `content_hash` , tabel `questions` , `question_options` , `question_assets` , `question_reviews` , `prediction_runs` .
- ❑ **Task 2.4 (Central API — Exam Packages & Envelope Encryption):** Implementasi `POST /packages` , `/build` (worker async), `/publish` (Ed25519 sign + checksums SHA-256), struktur folder `manifest.json` , `questions.json` , `assets/` , `answer-key.enc` , `explanations.enc` , `scoring.json` , `checksums.json` , `signature.sig` . Rename tabel ke `exam_package_versions` , `exam_package_items` . Envelope encryption: AES-256-GCM + RSA-OAEP per edge.
- ❑ **Task 2.5 (Central API — Distributions & Edge Heartbeat):** Implementasi `POST /packages/{id}/distribute` , `GET /packages/{id}/distributions` , `POST /edges/register` , `POST /edges/heartbeat` , `GET /edges/{id}/pending-packages` , `POST`

- `/edges/{id}/packages/{packageId}/acknowledge` . Nonce + HMAC signature, Redis distributed lock per sekolah.
- ❑ **Task 2.6 (Central API — Sync Results Idempotent):** Implementasi `POST /sync/results` (batch maks. 200 attempt, `ON CONFLICT DO NOTHING`, `uniqIdem`), `POST /sync/results/{batchId}/acknowledge` . Simpan `sync_batches`, `sync_events`, `central_attempts`, `central_attempt_answers`, `central_results` . Materialized view `central_results_mv` untuk rekap nasional.
 - ❑ **Task 2.7 (Edge Server — Bootstrap & DB):** Setup `apps/edge-server` Go single binary untuk Windows & Ubuntu. Drizzle→SQL migration untuk tabel edge Bab 10 (`edge_configuration`, `local_users`, `package_cache`, `package_files`, `local_exam_sessions`, `local_attempts`, `local_attempt_answers`, `local_results`, `result_outbox`, `sync_checkpoints`, `edge_audit_logs`, `edge_notifications`, `local_participants`, `exam_tokens`). Dukungan PostgreSQL produksi + SQLite WAL prototipe.
 - ❑ **Task 2.8 (Edge Server — Local API & Token Sekali Pakai):** Implementasi endpoint Bab 6 lokal: `/local/v1/auth/login`, `/local/v1/exams`, `/local/v1/exams/{id}/start` (aktivasi sesi dengan optimistik lock anti dua operator), `/local/v1/exams/{id}/attempts` (klaim **token sekali pakai** `SNT-XXXX-XXXX-XXXX`, single-use di dalam transaction), `/local/v1/attempts/{id}/questions` (tanpa kunci jawaban), `/local/v1/attempts/{id}/answers/{questionId}` (autosave debounced), `/local/v1/attempts/{id}/submit` (koreksi otomatis di edge + skor instan), `/local/v1/attempts/{id}/result`, `/local/v1/health`, `/local/v1/readiness` .
 - ❑ **Task 2.9 (Edge Server — Protokol SentraExam Client) — Owner: Backend Edge + Tim SentraExam Client:** Implementasi protokol client-edge milik sendiri sesuai Bab 6.H: bootstrap, registrasi instance, klaim token, pengambilan soal, autosave idempoten berbasis revisi, recovery, final submit dengan receipt, result, serta WebSocket `/ws/exam` . Definisikan `ExamClientGateway` dan implementasi `LocalExamClientGateway` di `apps/edge-server/internal/examclient/` ; dokumentasikan versioning, error code, payload, retry, dan compatibility matrix di `docs/exam-client/PROTOCOL-SPEC.md` .
 - ❑ **Task 2.10 (Edge Server — Sync Worker & Outbox):** Implementasi sync worker edge → pusat dengan batch, exponential backoff (5s→30s→2m→10m→1h), checkpoint (`sync_checkpoints`), idempotency key per attempt. Data gagal tidak boleh hilang; retry maksimum 12 kali lalu `FAILED_PERMANENT` + notifikasi ke `/local/notifications` .
 - ❑ **Task 2.11 (WebSocket Monitor LAN):** Implementasi WebSocket hub (`edge-server/internal/ws`) di `EDGE_WS_PATH=/ws/monitor` . Broadcast event peserta (`ACTIVE`, `SUBMITTED`, `DISCONNECTED`) ke klien LAN. Hook ke `packages/ui` + React `useWebSocket` . Update halaman `/local/monitor` & `/mobile/monitor` untuk konsumsi data real.
 - ❑ **Task 2.12 (In-App Notification Edge):** Implementasi tabel `edge_notifications` + endpoint `GET /local/v1/notifications`, `POST /local/v1/notifications/{id}/read` . Integrasi Sonner di edge-admin-web dan panel notifikasi di mobile. Trigger: paket diverifikasi, sesi aktif, peserta selesai, sinkronisasi sukses/gagal, storage kritis.
 - ❑ **Task 2.13 (Frontend Data Binding):** Ganti seluruh data dummy dari Tahap 1 dengan TanStack Query (Central Web, Edge Admin Web, Mobile) + WebSocket realtime. Semua mutasi memakai Server Actions / fetch ke Central API / Edge API sesuai Bab 6.
 - ❑ **Task 2.14 (Seed Data 3 Sekolah):** Script `seed/central.sql` & `seed/edge.sql` untuk SMAN 1 Nusantara (`SCH-JKT-001`), SMKN 2 Bahari (`SCH-SBY-002`), SMA Tunas Bangsa (`SCH-MKS-003`); soal dummy `APPROVED` (7 subtes × 20 soal), 5 soal `AI_DRAFT`, 1 paket `utbk-2026-set-01 v1.2.0` . Data peserta dummy 3 sekolah.

Tahap 3: Hardening Keamanan, Automated Tests, Docker Compose, Load Test, Deployment, Dokumentasi

Tujuan: Menyempurnakan sistem menjadi production-ready dengan pengujian menyeluruh, ketahanan, dan rilis ke server pusat & edge.

- ❑ **Task 3.1 (Automated Tests Lengkap) — Owner: QA, dibantu semua tim aplikasi:** Go: `unit` (envelope encryption, token sekali pakai, idempotency), `integration` (edge→central sync, distribusi paket, verifikasi signature). React: Vitest + Testing Library untuk halaman kritis. Tauri/Rust: unit test command dan storage encryption. E2E: Playwright/WebDriver menguji alur vertical slice lengkap (admin buat paket → edge download → SentraExam Client mengerjakan → recovery koneksi → skor instan → sinkron ke pusat). Target coverage ≥ 75% backend, ≥ 60% frontend/client.
- ❑ **Task 3.2 (Ketahanan & Resilience):** Implementasi skenario Bab 7: edge restart saat ujian, peserta refresh, paket rusak (fallback verifikasi), hasil duplikat, sync putus di tengah, waktu komputer peserta tidak sesuai, storage hampir penuh (auto-cleanup paket lama dengan konfirmasi), edge belum unduh paket saat jadwal (blocker + `edge_notifications`), dua operator aktifkan sesi (optimistic lock `version`), pusat down (edge tetap jalan & retry). Tambahkan `pre-exam readiness check` 15 menit sebelum jadwal.

- ❑ **Task 3.3 (Non-Functional — SEO & Security Hardening) — Owner: Security + Infrastructure:** Dynamic `<title>`, meta description, OG tags, canonical di central-web + `robots.txt` (`noindex` untuk `/local`). CSP header, HSTS, XSS sanitization, CSRF untuk form pusat, validasi sisi server, dependency audit JS/Go/Rust tanpa high/critical, signed installer dan signed update manifest SentraExam Client, rate limit lanjutan, dan seluruh secret dari secret manager/environment injection.
- ❑ **Task 3.4 (Docker Compose Central & Edge):** Buat `deployments/central/docker-compose.yml` (postgres, redis, minio, central-api, central-worker, central-web, nginx/caddy, prometheus, grafana) dan `deployments/edge-linux/docker-compose.yml` + `deployments/edge-windows/install.ps1` (edge-server, edge-postgres, edge-admin-web, caddy LAN). Verifikasi `docker compose up` sukses di kedua sisi.
- ❑ **Task 3.5 (Load Test Terpisah):** Central: k6 skenario 1.000 edge heartbeat bersamaan + 1.000 sekolah sync 200 attempt/batch bersamaan (target p95 < 800ms, error < 0.5%). Edge: k6 skenario 200 peserta autosave 1 jawaban/2 detik selama 195 menit (target p95 < 150ms). Output laporan HTML disimpan di `docs/operations/load-test/`.
- ❑ **Task 3.6 (Simulasi Minimal 3 Sekolah End-to-End):** Jalankan 3 edge server (`EDGE-JKT-001` , `EDGE-SBY-002` , `EDGE-MKS-003`) bersamaan; distribusi 1 paket ke ketiganya; 30 instance SentraExam Client per sekolah mengerjakan ujian; uji putus LAN singkat, restart client, recovery attempt, submit receipt, dan sinkronisasi hasil ke pusat; verifikasi `central_results_mv` menampilkan 90 attempt lengkap tanpa duplikasi.
- ❑ **Task 3.7 (Dokumentasi Instalasi & Operasional):** Tulis `docs/architecture/OVERVIEW.md` , `docs/api/openapi.yaml` (generate dari Go), `docs/security/THREAT-MODEL.md` , `docs/operations/CENTRAL-INSTALL.md` , `docs/operations/EDGE-INSTALL-WINDOWS.md` , `docs/operations/EDGE-INSTALL-LINUX.md` , `docs/operations/OPERATOR-RUNBOOK.md` , `docs/exam-client/PROTOCOL-SPEC.md` .
- ❑ **Task 3.8 (Production Build & Deployment):** Verifikasi `go build -o bin/*` (central-api, central-worker, edge-server) sukses lintas Windows amd64 & Linux amd64. Verifikasi `pnpm build` (central-web, edge-admin-web, exam-client) & `expo build` (mobile). Deploy ke VPS pusat (Docker Compose) + publikasi edge-server binary via URL internal Tailscale. Tag rilis `v1.0.0-mvp` .
- ❑ **Task 3.9 (Laporan Keterbatasan & Roadmap Lanjutan):** Tulis `docs/operations/LIMITATIONS.md` yang menyebutkan: mode kiosk MVP belum menjadi anti-cheating OS absolut, ML/DL pipeline masih skeleton, data warehouse belum terpisah, mobile peserta belum ada, dan proctoring AI belum ada. Roadmap v1.1: hardening kiosk per OS, staged auto-update SentraExam Client, ML pipeline produksi, ClickHouse DW, mobile peserta, dan SSO sekolah.

18. Struktur Tim dan Pembagian Tanggung Jawab

Bab ini menetapkan pembagian peran, batas tanggung jawab, alur koordinasi, dan keluaran kerja untuk membangun SentraUjian secara terukur.

A. Prinsip Kerja Tim

1. Setiap pekerjaan memiliki satu pemilik utama, dependensi yang jelas, kriteria penerimaan, dan bukti verifikasi.
2. Perubahan lintas tim harus melalui kontrak tertulis, pull request, dan review dari pemilik komponen terkait.
3. Tim bekerja bertahap sesuai Tahap 1, Tahap 2, dan Tahap 3 pada Bab 12. Tidak ada pekerjaan yang melompati dependensi.
4. Keputusan arsitektur, perubahan skema data, dan perubahan kontrak API dicatat dalam dokumentasi proyek.

B. Product Manager

- Menetapkan tujuan produk, prioritas, ruang lingkup, dan urutan pekerjaan.
- Menjaga agar kebutuhan sekolah, operator, peserta, dan administrator tercermin dalam PRD.
- Menyusun acceptance criteria bersama QA dan pemilik teknis.
- Mengelola backlog, risiko, keputusan produk, dan komunikasi lintas tim.
- Memimpin review milestone dan memastikan setiap tahap memiliki keputusan go atau no-go.

C. Backend Engineer

- Membangun Backend Central dan Backend Edge sesuai kontrak API serta event.
- Mengimplementasikan autentikasi, otorisasi, token ujian sekali pakai, scoring di edge, sinkronisasi idempoten, dan audit log.
- Menjaga validasi input, error response, observability, migration, dan backward compatibility.
- Menyediakan unit test, integration test, mock server, dan dokumentasi endpoint.

D. Frontend Engineer

- Membangun Central Web, Edge Admin Web, dan antarmuka operasional berdasarkan design system.
- Mengimplementasikan state loading, empty, error, success, permission, dan offline secara nyata.
- Mengintegrasikan API dengan tipe data yang konsisten dan menjaga aksesibilitas keyboard.
- Memastikan layout responsif untuk desktop, tablet, dan ponsel tanpa horizontal overflow.

E. QA Engineer

- Menyusun test plan, test case, regression suite, dan acceptance checklist untuk setiap task.
- Menguji alur normal, offline-LAN, koneksi terputus, retry, duplikasi event, permission, dan pemulihan sesi.
- Menjalankan automated test, API test, integration test, E2E test, load test, serta validasi visual.
- Mencatat bug dengan langkah reproduksi, dampak, severity, evidence, dan hasil retest.

F. Infrastructure dan Server Engineer

- Menyiapkan server Central, Edge Server sekolah, jaringan LAN, reverse proxy, DNS, TLS, firewall, dan backup.
- Membangun Docker Compose, CI/CD, environment management, health check, log aggregation, monitoring, dan alerting.
- Menjaga deployment dapat diulang, memiliki rollback, dan tidak mengekspos secret ke source code.
- Menguji kapasitas, resource limit, disaster recovery, dan prosedur pemulihan layanan.

G. AI/ML Engineer

- Menentukan batas penggunaan AI/ML pada prediksi soal, klasifikasi, dan analitik.
- Membangun kontrak pipeline, dataset protocol, evaluasi, versioning model, dan monitoring kualitas.
- Memastikan fitur berbasis prediksi memiliki status yang jelas dan wajib melalui review manusia sebelum digunakan.
- Menjaga privasi data, reproducibility, auditability, dan dokumentasi keterbatasan model.

H. Administrasi dan Dokumentasi

1. Administrasi Proyek

- Mengelola jadwal rapat, notulen, daftar keputusan, agenda review, dan tindak lanjut.
- Menjaga struktur folder proyek, daftar dokumen, versi file, dan status persetujuan.
- Membantu Product Manager menyiapkan laporan milestone, risiko, dan kebutuhan koordinasi.

2. Dokumentasi Teknis

- Menulis dan merawat panduan instalasi, konfigurasi, operasi, troubleshooting, dan handoff.
- Mengubah hasil keputusan teknis menjadi ADR, runbook, changelog, dan dokumentasi onboarding.
- Memastikan dokumentasi sesuai implementasi aktual dan diverifikasi sebelum release.

Peran administrasi dan dokumentasi merupakan bagian penting dari delivery. Keduanya memiliki akses pada informasi proyek sesuai kebutuhan kerja dan mengikuti standar keamanan serta kerahasiaan yang sama dengan seluruh anggota tim.

I. Koordinasi dan Definition of Done

1. Setiap task mencantumkan owner, reviewer, dependency, deliverable, test, dan acceptance criteria.
2. Task dinyatakan selesai setelah kode, test, dokumentasi, observability, dan review tersedia sesuai ruang lingkupnya.
3. Setelah satu tahap selesai, Product Manager memimpin review bersama Backend, Frontend, QA, Infrastructure, AI/ML, dan Administrasi/Dokumentasi.
4. Tahap berikutnya hanya dimulai setelah laporan hasil, daftar risiko, dan keputusan kelanjutan disetujui.

PRD v2.0.0

Blueprint operasional jaringan lokal, Edge Server, BYOD, SentraExam Client, dan governance hasil tes berdasarkan perumusan kebutuhan dan validasi arsitektur.

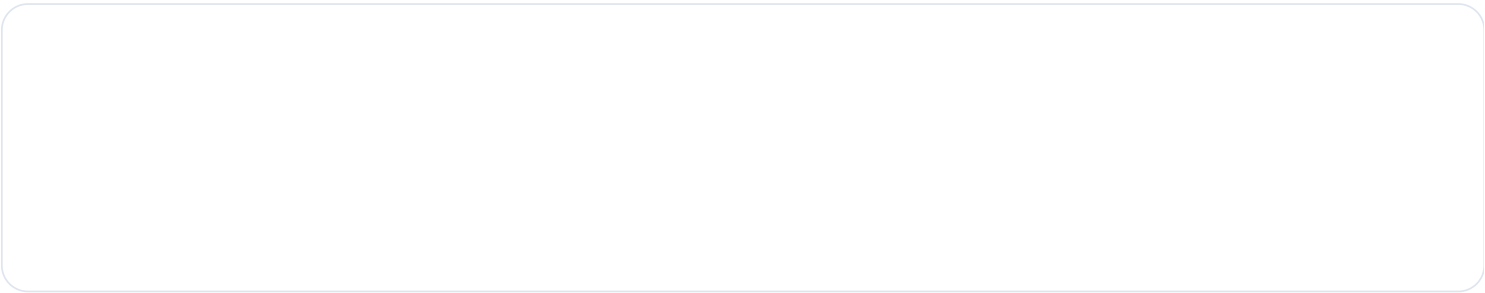
19. Model Produk v2

Versi 2 memperluas SentraUjian dari platform ujian sekolah menjadi platform assessment yang dapat digunakan untuk TPA, CPNS, seleksi dosen, UTBK, tryout, dan kebutuhan assessment lainnya. Pelaksanaan tes menggunakan Edge Server di lokasi, sedangkan server pusat mengelola sesi, paket, peserta, sinkronisasi, rekonsiliasi, dan laporan resmi.

19.1 Prinsip akses

- Peserta dan Edge Server berada pada jaringan lokal yang sama, melalui Wi-Fi atau LAN.
- Internet tidak menjadi syarat untuk mengerjakan tes setelah data persiapan tersinkron.
- Token operator mengaktifkan satu sesi tes tertentu, bukan akses umum ke sekolah.
- Peserta masuk dengan nomor peserta dan PIN pribadi.
- Seluruh peserta dalam satu sesi menggunakan aturan hasil dan scoring yang sama.

20. Arsitektur Central-Edge v2

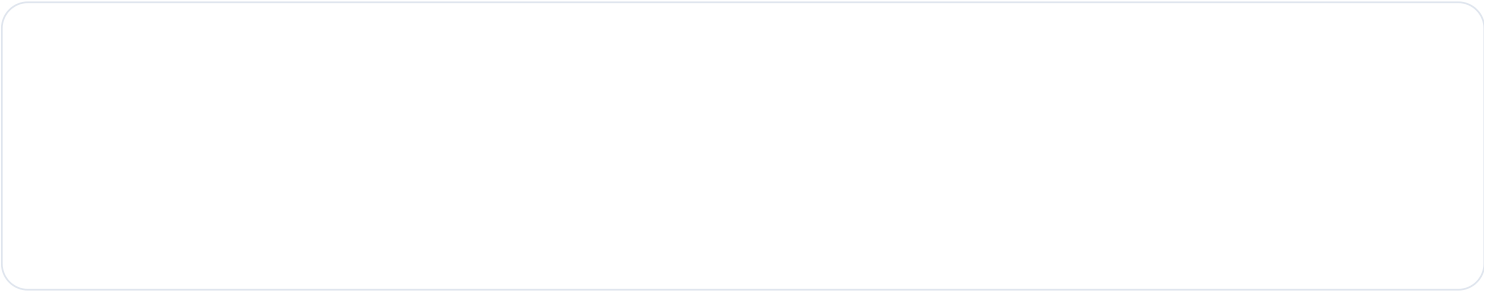


20.1 Pembagian tanggung jawab

Komponen	Tanggung jawab	Batasan
Central Platform	Master data, sesi, roster, paket, approval, sinkronisasi, rekonsiliasi, laporan resmi	Tidak menjadi jalur wajib selama pengerjaan lokal
Edge Server	Validasi token, autentikasi lokal, serve soal, autosave, scoring, recovery, outbox	Hanya untuk sekolah dan subnet yang tervalidasi
SentraExam Client	Kiosk, readiness check, koneksi lokal, timer display, reconnect	Proteksi BYOD memiliki batas berdasarkan OS

Komponen	Tanggung jawab	Batasan
Router/Firewall	Kontrol jaringan peserta dan pembatasan akses internet	Tidak diambil alih otomatis oleh Edge Server

21. Siklus Sesi dan Gate Aktivasi



21.1 Gate wajib sebelum aktivasi

1. Subnet lokal telah divalidasi dan disetujui admin pusat.
2. Paket dan kunci jawaban memiliki versi, checksum, dan signature valid.
3. Seluruh peserta berstatus wajib sudah lulus readiness, kecuali admin pusat mengubah statusnya.
4. Seluruh perangkat telah terdaftar, simulasi lulus, dan memakai versi client minimum.
5. Kapasitas Edge Server mencukupi.
6. Operator lokal berhasil login dan token cocok dengan sesi.
7. Waktu berada di dalam jendela aktivasi.

22. Roster, Perangkat, dan BYOD

- Admin pusat membuat roster utama, nomor peserta, PIN, dan penetapan sesi.
- Operator dapat mengajukan peserta tambahan, tetapi persetujuan pusat wajib sebelum sinkronisasi.
- Operator dapat memperbaiki data teknis dan mendaftarkan perangkat sebelum sesi aktif.
- Perangkat pribadi wajib menjalankan simulasi sebelum hari tes.
- Perangkat pengganti hanya boleh didaftarkan sebelum sesi aktif.
- Setelah attempt dimulai, perpindahan perangkat tidak diperbolehkan.
- Perubahan IP lokal tidak memaksa pendaftaran ulang selama perangkat masih berada pada jaringan sekolah dan dapat menjangkau Edge Server.

23. Aturan Pengerjaan dan Scoring

Area	Keputusan v2
Jenis soal	Seluruh soal berupa pilihan ganda dan wajib memiliki tepat satu kunci.
Randomisasi	Urutan soal dan pilihan diacak per peserta menggunakan manifest atau seed yang dapat diaudit.
Navigasi	Peserta dapat kembali, mengubah jawaban, dan menandai soal selama waktu tersedia.
Scoring	Benar mendapat bobot; salah dan kosong mendapat nol; tidak ada pengurangan nilai.
Waktu	Timer resmi berasal dari Edge Server dan tetap berjalan saat koneksi client terputus.
Selesai	Waktu habis mengunci jawaban otomatis; selesai lebih awal memakai konfirmasi dua langkah.

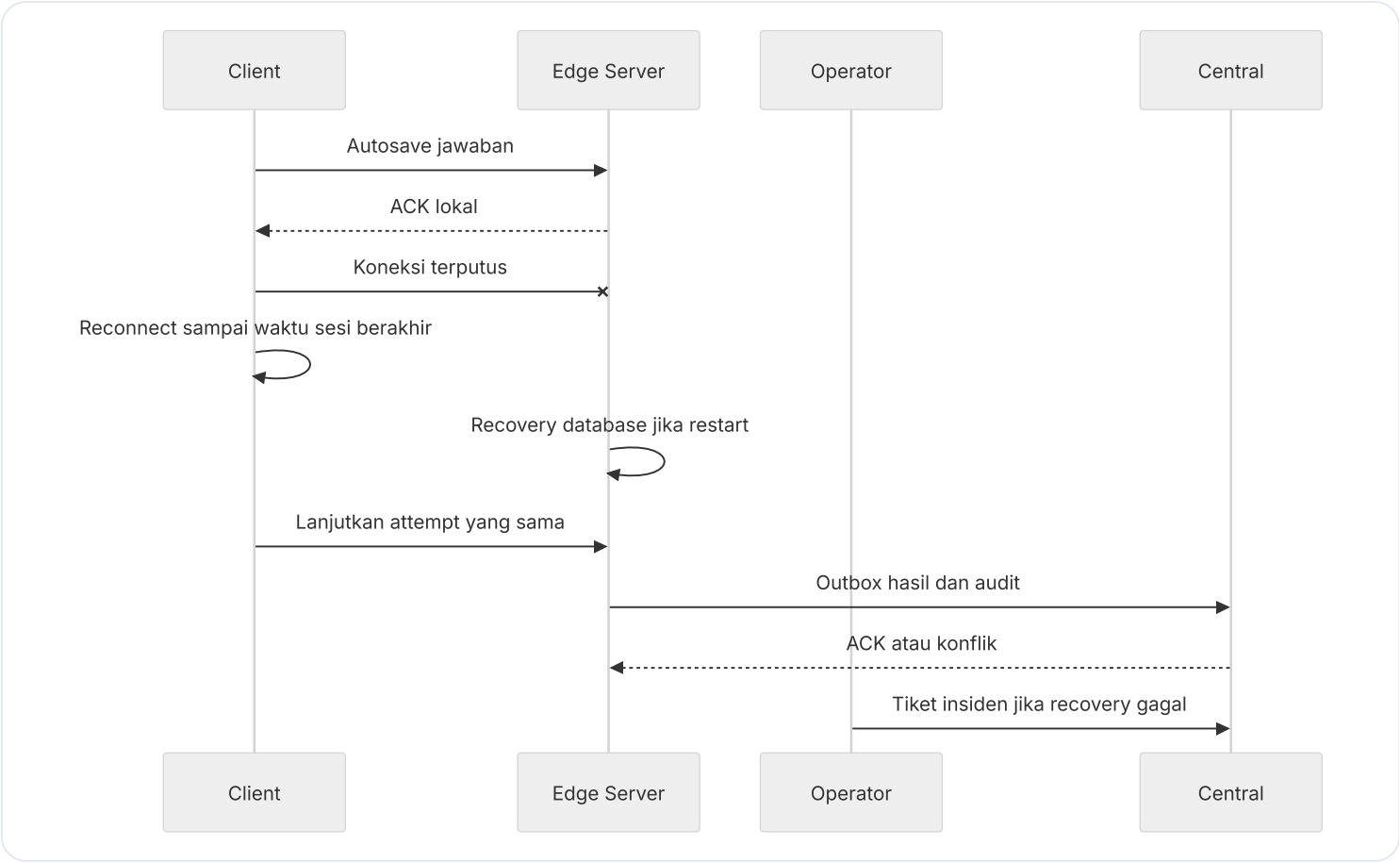
24. Mode Tampilan Hasil

Admin pusat menetapkan daftar mode yang diizinkan. Operator memilih satu mode sebelum sesi aktif. Satu sesi hanya memiliki satu mode untuk seluruh peserta.

Mode	Isi tampilan pertama
Mode 1	Skor dan ringkasan jumlah benar, salah, kosong, durasi, serta status jika dikonfigurasi.
Mode 2	Isi Mode 1 ditambah jawaban peserta, status benar/salah, dan jawaban benar tanpa pembahasan.
Mode 3	Isi Mode 2 ditambah pembahasan lengkap dan rekomendasi materi jika tersedia.

Hasil peserta hanya tampil satu kali. Peserta tidak dapat mencetak atau mengunduh hasil. Bukti keikutsertaan diterbitkan terpisah oleh operator atau admin melalui laporan resmi.

25. Gangguan, Recovery, dan Insiden



- Jawaban terakhir yang berhasil disimpan dipertahankan.
- Timer tetap berjalan berdasarkan Edge Server.
- Jika Edge Server mati lama, admin pusat menentukan keputusan berdasarkan bukti insiden.
- Jika paket sudah terlihat oleh satu peserta lalu sesi gagal, paket diblokir dan sesi pengganti memakai paket baru atau ekuivalen.
- Paket ekuivalen mengikuti blueprint, lalu direkomendasikan AI/ML dan disetujui reviewer akademik.
- Persetujuan darurat admin pusat boleh digunakan bila reviewer tidak tersedia; hasil tetap sah dan tidak diubah retroaktif.

26. Sinkronisasi, Finalisasi, dan Rekonsiliasi

- Outbox memakai retry, checksum, idempotency key, dan ACK.
- Konflik koreksi lokal diselesaikan dengan data pusat sebagai keputusan final.
- Operator dapat mengajukan koreksi baru berdasarkan versi pusat terbaru selama batas koreksi masih terbuka.
- Admin pusat menentukan batas finalisasi per sesi.
- Setelah batas tercapai, sesi dapat menjadi **Final dengan Pengecualian**.
- Data terlambat masuk area rekonsiliasi dan tidak mengubah hasil final secara otomatis.
- Status sesi tetap **Final dengan Pengecualian** setelah rekonsiliasi.
- Laporan resmi baru dapat diterbitkan setelah semua data terlambat memiliki keputusan.

27. Jaringan, DNS, dan Akses

- Jalur peserta mendukung IP lokal, hostname lokal, dan QR Code.
- Edge Server tidak menjalankan DHCP pada jaringan sekolah biasa.
- DHCP Edge hanya boleh aktif jika operator memilih jaringan khusus dan sistem mendeteksi tidak ada DHCP lain.
- Subnet resmi dipilih operator melalui Network Readiness Check lalu disetujui admin pusat.
- Jika subnet berubah pada hari tes dan belum disetujui pusat, sesi tidak dapat diaktifkan.
- Firewall sekolah menjadi kontrol utama akses internet dan komunikasi antarpeserta; Edge Server hanya memverifikasi hasilnya.
- Satu URL verifikasi resmi dapat memakai split-horizon DNS; sertifikat Edge unik dibuat dari private key yang tetap berada di Edge.

28. Operasional, Monitoring, dan Support

- Dashboard Edge menampilkan status server, storage, database, perangkat, kapasitas, backup, autosave, sinkronisasi, dan konflik.
- Backup otomatis terenkripsi dibuat lokal dan disinkronkan ke pusat saat internet tersedia.
- Kapasitas ditentukan dari benchmark; sistem menolak peserta tambahan ketika batas tercapai.
- Jika roster melebihi kapasitas, admin pusat membagi sesi atau menyediakan Edge tambahan.
- Operator menangani masalah dasar melalui panduan lokal.
- Masalah lanjutan menjadi tiket insiden berisi log, timestamp, peserta terdampak, dan evidence otomatis.

29. Riwayat Perubahan v2

Versi	Tanggal	Ringkasan
v2.0.0	22 September 2026	Menambahkan model assessment multi-keperluan, Edge Server lokal Wi-Fi/LAN, BYOD, SentraExam Client, readiness gate, scoring, mode hasil, recovery, rekonsiliasi, laporan, dan verifikasi bukti.
v1.0.0	Sebelumnya	Versi dasar PRD sebelum perumusan kebutuhan dan validasi arsitektur.